

CONVERGENCE AND PERFORMANCE RESULTS FOR REINFORCEMENT LEARNING WITH TWO TIME SCALES

Diogo Filipe de Sousa Carvalho

Instituto Superior Técnico, University of Lisbon
diogo.s.carvalho@tecnico.ulisboa.pt

THESIS PROPOSAL

Thesis Committee:

Gergely Neu

João Manuel de Freitas Xavier

Pedro Alexandre Simões dos Santos

Francisco António Chaves Saraiva de Melo

September 11, 2023

Abstract

This thesis investigates reinforcement learning methods with function approximation that are asymptotically well behaved, i.e., that provably converge to a good solution. We build on two time-scale stochastic approximation algorithms to design our proposed methods. In two time-scale stochastic approximation algorithms, two different sets of parameters are iteratively updated at two different time scales, i.e., one set of parameters is updated at a much faster time scale than the other. The use of different time scales greatly facilitates the analysis of the algorithms: when analyzing the parameters that are updated at a faster time scale, the other set of parameters can be treated as non-changing; conversely, when analyzing the parameters that are updated at a slower time scale, the other set of parameters can be treated as having converged.

The thesis starts by analyzing the standard case of reinforcement learning with linear function approximation, where state-action features are stationary and defined prior to learning. In our first contribution, we propose regularized alternatives to Q -learning-based methods and show that the proposed regularization techniques ensure convergence of the algorithms. We then propose to extend our results by showing that these regularized variants of Q -learning also converge when combined with non-linear non-stationary features, as long as those features can be shown to converge at a slower time scale, which can be done through supervised or semi-supervised learning updates that do not affect the reinforcement learning algorithm. As a last step in this line of work, we intend to investigate end-to-end reinforcement learning methods that are compliant with non-linear function approximation, analyzing the convergence of the regularized versions of Q -learning previously considered with neural network architectures. Building on the same ideas of two time-scale stochastic approximation, we will conclude our thesis by investigating the convergence of Q -learning algorithms in non-stationary settings by adopting a perspective of hierarchical reinforcement learning, by revisiting full-gradient reinforcement learning methods that may provide guarantees of convergence to better limit points than existing residual-based approaches.

Summary

1	Introduction	3
1.1	Expected contributions	3
1.2	Document outline	5
2	Background	6
2.1	Stochastic approximation	6
2.2	Markov decision processes	8
2.3	Reinforcement learning	10
2.4	Divergence of Q -learning	13
3	Related work	18
3.1	Linear function approximation	18
3.2	Non-linear function approximation	19
3.3	Policy-based methods	20
3.4	Learning with non-stationarities	21
3.5	Full-gradient methods	21
4	Linear function approximation	22
4.1	Q -learning	22
4.2	Approximate policy iteration	24
4.3	Discussion	25
5	Learning with non-stationarities	27
5.1	Learning non-linear features as non-stationarity	27
5.2	Hierarchical Markov decision processes as non-stationarity	28
6	Future work	31
6.1	Between gradients and semi-gradients	31
6.2	Multi-Bellman operator	32
7	Conclusion	35

1 Introduction

In reinforcement learning, an agent interacts with an environment in the form of a stochastic process where, at each time step, the environment is at a certain state, the agent performs an action, the environment changes to a new state and the agent receives a reward. The goal of the reinforcement learning agent is to approximate the value of performing each action in each state, where by value we mean the optimal expected sum of time-discounted rewards. The importance of this task cannot be overstated, as an accurate value function enables an agent to make optimal decisions by selecting actions with the highest value in a given state [47]. The value function also facilitates environment evaluation and enables comparisons between different environments. If we can use a tabular representation for the value function, meaning that we can store the value of performing each action on each state individually, the Q -learning algorithm converges to the correct value function [59]. When it is not possible or desirable to store values in a table, for example when there are too many states or actions, Q -learning can be combined with function approximation [37]. Unfortunately, the combination of Q -learning and function approximation is troublesome. Even when the function approximation space is linear, the algorithm is not guaranteed to converge [54] and there are examples of divergence [4, 57]. In fact, the approximation problem that Q -learning addresses does not have, in general, a solution [38]. This thesis investigates reinforcement learning methods that are asymptotically well behaved, i.e., that converge to a solution and the solution is “good”. We make our goal explicit:

Research question How can we approximate the optimal state-action value function?

For our purposes, we mostly build on the ideas of two time-scale stochastic approximation to construct the proposed methods. In two time-scale stochastic approximation, two different sets of parameters are iteratively updated along two different time-scales. Specifically, one set of parameters is updated much slower than the other. The difference between time-scales allows us to consider that the parameters that change slower are static and that the parameters that change faster already converged. In accordance with this insight, we can decouple the convergence analysis of the two sets of parameters that are learning concurrently. For the convergence proofs, we rely on well established theoretical machinery from two time-scale stochastic approximation. At times, we do extend existing results, tailoring them to our specific technical needs whenever necessary.

1.1 Expected contributions

With the presently proposed thesis, we expect to contribute to the problem of convergence of reinforcement learning methods in multiple settings using ideas and techniques from stochastic approximation. Our contributions consider three settings. We start by providing regularized variants of two Q -learning methods to be used with (i) linear function approximation. We then enlarge our scope to two (ii) non-stationary settings. We already established the contributions in these first two settings. In the continuation of this research, we intend to contribute with reinforcement learning methods that can be used with (iii) non-linear function approximation architectures, specifically neural networks, following two research directions. In the following, we detail each expected contribution in each setting.

1.1.1 Linear function approximation

First, we consider the setting of reinforcement learning with linear function approximation, where Q -learning, an off-policy method, and its on-policy counterpart SARSA, may diverge. For each of the methods, we propose an alternative, proving its convergence. Both methods comprise two sets of parameters that are iteratively updated along two different time scales.

Q -learning with linear function approximation We propose CQL, for “coupled Q -learning”. The method is inspired by the successful deep Q -network [40] and it may be casted as a regularized variant of Q -learning. Along the faster time-scale, the parameters of a state-action value function are updated to match the values computed from a target network. Along the slower time-scale, the parameters of the target network are updated so that it approximates the value of the regularized main network. We show convergence with probability 1. A version of this work was published at the proceedings of Advances in Neural Information and Processing Systems 33 (NeurIPS 2020).

Approximate policy iteration with linear function approximation We propose 2TAP, for “two time-scale approximate policy iteration”. The parameters learning along the faster time-scale parameterize a linear function approximation architecture for a regularized state-action value function. The parameters updated along the slower time-scale parameterize a policy and approximate a Boltzmann-like distribution over the perceived state-action values. We prove our proposed method converges with probability 1.

1.1.2 Learning with non-stationarities

Here, we extend existing results of convergence of Q -learning to two non-stationary settings. First, we build on a previously established convergence result for Q -learning with linear function approximation and show that the result still holds with non-stationary but convergent features. Then, we extend the convergence of Q -learning with tabular representations for non-stationary but convergent Markov decision processes.

Q -learning with non-linear non-stationary features We start by framing non-linear function approximation architectures for Q -learning as neural networks where the final linear layer is learned at the same time that non-linear hidden layers are learned. We consider that the final layer is being updated much faster than the hidden layers; we further assume it is being updated according to a regularized version of Q -learning. We show that if the hidden layers converge along a slower time-scale, the final layer also converges.

Hierarchical reinforcement learning We establish convergence of Q -learning in the non-stationary setting of hierarchical reinforcement learning with tabular representations. We start by formalizing feudal hierarchies in reinforcement learning as a pair of Markov decision processes where the high-level Markov decision process depends on the policy executed at the low-level. We show that the hierarchy is stable, i.e., that if the low-level policy converges, the high-level Markov decision process converges. Finally, we show convergence of Q -learning. A version of this work will be published at the proceedings of the 26th European Conference on Artificial Intelligence (ECAI 2023).

1.1.3 Future work

In the continuation of this research we expect to pursue two more contributions. As a first contribution, we propose to build a convergent reinforcement learning method with non-linear function approximation by exploiting already existing ideas on full-gradient methods. As the second contribution, we propose to build a convergent reinforcement learning method with linear function approximation, building on novel ideas that extend the Bellman operator and the Q -learning algorithm.

Between gradients and semi-gradients We intend to revisit full-gradient descent methods for reinforcement learning [4], in the context of non-linear function approximation. We know that reinforcement learning methods that perform semi-gradient descent on the Bellman error, such as Q -learning, can diverge when combined with function approximation. Methods that perform full-gradient descent on the other hand, necessarily converge. However, convergence of the latter may happen to undesirable local minima, with great impact on the quality of the approximation and its usefulness to control the agent. We investigate full-gradient reinforcement learning methods that may converge to global optima.

Multi-Bellman operator Finally, we intend to propose an alternative algorithm that effectively solves the function approximation problem, unlike the original Q -learning. Both the problem and the algorithm proposed can be seen as extensions of the original, in the sense that we propose the multi-Bellman operator from which we intend to build the multi Q -learning algorithm. We show that, in the linear function approximation case, the projected multi-Bellman operator is contractive, guaranteeing the existence of a unique solution. We intend to propose a stochastic approximation algorithm that converges to such solution. We expect the results to extend to non-linear function approximation.

1.2 Document outline

We start the exposition with background knowledge in Section 2, dividing it between stochastic approximation, Markov decision processes and reinforcement learning. Section 3 discusses work related to our own. Afterwards, we present each expected contribution of the thesis in detail. Contributions on the convergence of reinforcement learning with linear function approximation appear in Section 4. One of the proposed methods learns off-policy and appears in Section 4.1; the other learns on-policy and is presented in Section 4.2. Our results on linear function approximation in two non-stationary settings appear in Section 5. They are further divided in the settings of non-linear non-stationary features in Section 5.1 and non-stationary environments in Section 5.2. The future work forms Section 6, where we expand on our two expected contributions to non-linear function approximation. The line of research that follows full-gradient methods appears in Section 6.1; the line of research that follows the multi-Bellman operator appears in Section 6.2. We conclude in Section 7.

2 Background

In this section, we provide background on stochastic approximation, Markov decision processes and reinforcement learning. The goal of the exposition is not only to recover knowledge that is important for the discussions that follow, but also to establish notation to be used throughout the remainder of the document.

Notation We denote random variables (r.v.s) using upright letters, as in x or a . We use lowercase letters to denote functions, as in v or q , and calligraphic letters to denote sets, as in $\mathcal{X}$ or $\mathcal{A}$. Vectors are represented as bold lowercase letters. For example, $\mathbf{z}$ denotes a random vector and $\mathbf{z}$ an instance thereof. Operators are represented as upright uppercase letters such as H . Matrices are represented using bold uppercase letters, as in $\mathbf{M}$. If $\mathbf{z}$ is a random vector taking values in $\mathcal{Z}$ with distribution $\mu \in \Delta(\mathcal{Z})$, we use $\mathbf{z} \sim \mu$ or simply $\mathbf{z} \sim \mathbf{z}$ to denote that $\mathbf{z}$ is sampled according to the distribution μ and that we use $\mathbf{y} \sim \mu$ or just $\mathbf{y} \sim \mathbf{z}$ if $\mathbf{y}$ is a r.v. with the same distribution as $\mathbf{z}$. We write $\mathbb{E}_{x,a \sim \mu} [f(x, a)]$ or simply $\mathbb{E}_\mu [f(x, a)]$ to denote the expectation of f when the r.v.s x and a follow distribution μ .

2.1 Stochastic approximation

Robbins and Monro [48] introduced stochastic approximation to solve the equation

$$h(\mathbf{w}) = \mathbf{0}, \quad (1)$$

where $h : \mathbb{R}^K \rightarrow \mathbb{R}^K$. In stochastic approximation, we assume we do not know the function h and, consequently, cannot evaluate it exactly, regardless of the input. We assume we can, however, obtain on input $\mathbf{w} \in \mathbb{R}^K$ a noisy sample of $h(\mathbf{w})$. We write the noisy sample as $h(\mathbf{w}) + \mathbf{z}$, where $\mathbf{z}$ is a (sampled) noise random vector taking values in $\mathbb{R}^K$. To arrive at a solution $\mathbf{w}^*$ for (1), starting from an initial guess $\mathbf{w}_0$, the Robbins-Monro method iteratively updates its solution according to the rule

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (h(\mathbf{w}_t) + \mathbf{z}_{t+1}), \quad (2)$$

where α_t is a positive, real-valued, and decreasing step-size that we call the learning rate. The theoretical analysis of the stochastic approximation algorithm (2) relies on observing that the update in (2) resembles the Euler method to solve the ordinary differential equation

$$\dot{\mathbf{w}}_t = h(\mathbf{w}_t), \quad (3)$$

which consists of the update rule

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha h(\mathbf{w}_t). \quad (4)$$

More specifically, updates (2) and (4) differ only in the varying learning rates α_t and noise samples $\mathbf{z}_{t+1}$ that appear in the former. Following this remark, Borkar [9, Chapter 2] shows that if h is Lipschitz-continuous and the random variables $\{\mathbf{z}_{t+1}\}_{t \in \mathbb{N}_0}$ form a martingale-difference sequence, we can analyze the behaviour of the stochastic approximation update by analyzing the ordinary differential equation (3) instead. In particular, if (3) has a unique globally asymptotically stable equilibrium $\mathbf{w}^*$ and $\mathbf{w}_t$ is updated according to (2), $\mathbf{w}_t$ converges to $\mathbf{w}^*$ with probability 1 (w.p.1) [9, Chapter 2, Theorem 2].¹

¹We say that a random sequence of $\mathbf{w}_t \in \mathbb{R}^K$ converges to $\mathbf{w}^*$ w.p.1 if $\mathbb{P}[\lim_{t \rightarrow \infty} \mathbf{w}_t = \mathbf{w}^*] = 1$. Convergence w.p.1 is the notion of convergence used throughout the document, unless stated otherwise.

Alternatively to the single stochastic approximation scheme presented, we can also conceive the case where different components of an iteration are updated according to different learning rate schedules. Particularly, we are mostly interested in the case where, in addition to the main component being updated according to a sequence of learning rates $\{\alpha_t\}_{t \in \mathbb{N}_0}$, a second component is updated according to a sequence of learning rates $\{\beta_t\}_{t \in \mathbb{N}_0}$ such that $\beta_t = o(\alpha_t)$.² Intuitively, the second component is updated much slower. Considering this possibility gives rise to two time-scale stochastic approximation algorithms.

2.1.1 Two time-scale stochastic approximation

Consider an iterative algorithm that calls for another iterative sub-routine on each iteration. Traditionally, we would run the sub-routine until near-convergence and then use its output on the first routine. In such case, an outer and an inner loop would be in place. However, we can sometimes achieve the same effect if the two components are updated concurrently but on different time-scales. The sub-routine calls for a larger learning rate and iterates faster. Two time-scale stochastic approximation algorithms are composed of the iterations

$$\begin{aligned}\mathbf{v}_{t+1} &= \mathbf{v}_t + \alpha_t(f(\mathbf{v}_t, \mathbf{u}_t) + \mathbf{m}_{t+1}) \\ \mathbf{u}_{t+1} &= \mathbf{u}_t + \beta_t(g(\mathbf{v}_t, \mathbf{u}_t) + \mathbf{n}_{t+1}),\end{aligned}$$

where $\beta_t = o(\alpha_t)$, $f : \mathbb{R}^{K_1+K_2} \rightarrow \mathbb{R}^{K_1}$, $g : \mathbb{R}^{K_1+K_2} \rightarrow \mathbb{R}^{K_2}$ and $\mathbf{m}_t$ and $\mathbf{n}_t$ are sampled according to $\mathbf{m}_t \in \mathbb{R}^{K_1}$ and $\mathbf{n}_t \in \mathbb{R}^{K_2}$ that are noise random variables. The difference in the learning rates between the fast and slow iterations allows us to analyze two time-scale stochastic approximation algorithms as if the slow component is fixed and the fast component is in equilibrium throughout the execution of the algorithm. We can then use, as in the single time-scale stochastic approximation case, the stability of the corresponding ordinary differential equations to establish convergence of each update. We reproduce a convergence result for two time-scale stochastic approximation algorithms, which we use to establish most of our convergence results throughout this research. It is due to Borkar [9].

Theorem 1. *Consider the following assumptions:*

1. *f and g are both Lipschitz-continuous.*
2. *$\{\mathbf{m}_t\}_{t \in \mathbb{N}}$ and $\{\mathbf{n}_t\}_{t \in \mathbb{N}}$ are martingale-difference sequences with respect to the sigma-fields $\mathcal{F}_t = \sigma(\mathbf{v}_\tau, \mathbf{u}_\tau, \mathbf{m}_\tau, \mathbf{n}_\tau, \tau \leq t)$. Additionally, there exist $c_{\mathbf{m}}$ and $c_{\mathbf{n}}$ such that*

$$\begin{aligned}E[\|\mathbf{m}_t\|^2 \mid \mathcal{F}_t] &\leq c_{\mathbf{m}}(1 + \|\mathbf{v}_t\|^2 + \|\mathbf{u}_t\|^2), \\ E[\|\mathbf{n}_t\|^2 \mid \mathcal{F}_t] &\leq c_{\mathbf{n}}(1 + \|\mathbf{v}_t\|^2 + \|\mathbf{u}_t\|^2).\end{aligned}$$

3. *The learning rates satisfy $\beta_t = o(\alpha_t)$, $\sum_t \beta_t = \infty$ and $\sum_t \alpha_t^2 < \infty$.*
4. *For each $\mathbf{u} \in \mathbb{R}^{K_2}$, o.d.e. $\dot{\mathbf{v}}_t = f(\mathbf{v}_t, \mathbf{u})$ has a unique globally asymptotically stable equilibrium $\mathbf{v}^*(\mathbf{u})$, where $\mathbf{v}^* : \mathbb{R}^{K_2} \rightarrow \mathbb{R}^{K_1}$ is a Lipschitz-continuous function.*
5. *O.d.e. $\dot{\mathbf{u}}_t = g(\mathbf{v}^*(\mathbf{u}_t), \mathbf{u}_t)$ has a unique globally asymptotically stable equilibrium $\mathbf{u}^*$.*
6. *It holds that $\sup_t (\|\mathbf{v}_t\| + \|\mathbf{u}_t\|) < \infty$ w.p.1.*

Then, $(\mathbf{v}_t, \mathbf{u}_t) \rightarrow (\mathbf{v}^(\mathbf{u}^*), \mathbf{u}^*)$.*

²We say that $\beta_t = o(\alpha_t)$ if, for all $\epsilon > 0$, there exists t_0 such that, for all $t \geq t_0$, we have that $\beta_t < \epsilon \alpha_t$.

The assumptions that, in general, are harder to verify are Assumptions 4 and 5. These assumptions require existence and globally asymptotic stability of equilibrium points of o.d.e.s.. To verify these assumptions on applications, we can use Lyapunov’s second method [31] for o.d.e. stability, also known as the direct method, to establish the results. Finally, Assumption 6 demands that the iterates remain bounded. In most applications, the property is also not trivially established. However, a result from Bhatnagar [26] guarantees boundedness of two time-scale stochastic approximation iterates under conditions that are easier to establish, also resorting to global asymptotic stability of appropriate o.d.e.s.

We finish by noting that stochastic approximation is a method fit for various applications, due to its general formulation. In the specific case of reinforcement learning, there are sources of noise in the interaction of the agent with its environment. Those sources include the unavailability of the model, the stochasticity of the transitions, rewards and even of the action selection. At the same time, stochastic approximation allows us to analyze, theoretically, properties of the reinforcement learning algorithms such as asymptotic convergence. We formalize now the most common framework for reinforcement learning purposes.

2.2 Markov decision processes

A finite Markov decision process is a tuple $(\mathcal{X}, \mathcal{A}, \{\mathbf{P}_a\}_{a \in \mathcal{A}}, r)$, where $\mathcal{X}$ is a finite set of n states, called the state space; $\mathcal{A}$ is a finite set of m actions, called the action space and $\mathbf{P}_a$ defines, for each state, a probability distribution over next states;³ $r : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$ is the expected reward function. At time step t , the random variable $\mathbf{x}_t$ models the state of the process and the random variable $\mathbf{a}_t$ models the action taken by the agent. The random variable $\mathbf{r}_t$ models the reward received by the agent for taking action $a_t \sim \mathbf{a}_t$ in state $x_t \sim \mathbf{x}_t$ in a way that $\mathbb{E}[\mathbf{r}_t \mid \mathbf{x}_t = x, \mathbf{a}_t = a] = r(x, a)$. In our work we assume the expected reward in a bounded interval $[-r_{\max}, r_{\max}]$. After taking action a_t in state x_t , the environment transitions to a new state x_{t+1} according to the transition probabilities $\mathbf{P}_{a_t}(x_t)$.

A policy is an agent’s decision rule. The policy defines, for each state, a probability distribution over actions. We can, therefore, define that a policy is $\pi : \mathcal{X} \rightarrow \Delta(\mathcal{A})$.⁴ A single interaction of an agent with its Markovian environment thus generates a tuple of state, action, reward and next states (x_t, a_t, r_t, x'_t) . The Markov decision problem is the one of finding the policy that maximizes the expected sum of discounted rewards accumulated throughout the interaction of the agent with the environment, starting from a certain state. Formally, given a policy, we define the state value function $v^\pi : \mathcal{X} \rightarrow \mathbb{R}$ as

$$v^\pi(x) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}_t \mid \mathbf{x}_0 = x \right], \quad (5)$$

where the expectation is taken with respect to the sequence of actions chosen, states visited and rewards received, and $\gamma \in [0, 1[$ is a discount factor.⁵ The value function verifies the fixed-point equation:

$$v^\pi(x) = \mathbb{E} [\mathbf{r}_t + \gamma v^\pi(\mathbf{x}_{t+1}) \mid \mathbf{x}_t = x], \quad (6)$$

taking the expectation with respect to the action (implicit), reward and next-state. There exists a policy that maximizes the value of all states [47, Section 6.2]. We refer to it as

³We can represent $\mathbf{P}_a$ as an $n \times n$ matrix with component xx' given by $\mathbf{P}(\mathbf{x}_{t+1} = x' \mid \mathbf{x}_t = x, \mathbf{a}_t = a)$.

⁴We can represent π as an $n \times m$ matrix, with component xa given by $\mathbf{P}(\mathbf{a}_t = a \mid \mathbf{x}_t = x)$ if $a_t \sim \pi(x_t)$.

⁵The discount factor guarantees that the sum converges and is, intuitively, an inverse of inflation rate.

optimal policy and denote it as π^* and its state value function as v^* . We also define the state-action value function, or just value function, as the expected sum of discounted rewards accumulated throughout the interaction of an agent with its environment, starting from a certain state and choosing a certain action, as $q^\pi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$ such that

$$q^\pi(x, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid x_0 = x, a_0 = a \right],$$

where the expectation is taken with respect to the same quantities as before. We have that

$$v^\pi(x) = \mathbb{E} [q^\pi(x, a)],$$

taking the expectation with respect to the actions, and that the state-action value function verifies a fixed-point equation similar to (6). Specifically, we have that

$$q^\pi(x, a) = \mathbb{E} [r_t + \gamma q^\pi(x_{t+1}, a_{t+1}) \mid x_t = x, a_t = a], \quad (7)$$

where $a_{t+1} \sim \pi(\cdot \mid x_{t+1})$ and the expectation is with respect to state, action, reward, next state and next action. We refer to the problem of evaluating a policy π , i.e., computing v^π or q^π , as the prediction problem. Then, we refer to the state-action value function of an optimal policy π^* as q^* and we know it verifies the Bellman fixed-point equation

$$q^*(x, a) = \mathbb{E} \left[r_t + \gamma \max_{a' \in \mathcal{A}} q^*(x_{t+1}, a') \mid x_t = x, a_t = a \right], \quad (8)$$

taking the expectation with respect to the state, action, next state and reward. Knowing the optimal state-action value function implies knowing an optimal policy.

Remark 1. *Given the optimal state-action value function q^* for an MDP, the optimal policy π^* can be computed directly from q^* as $\pi^*(a \mid x) = 0$ if $a \notin \arg \max_{a' \in \mathcal{A}} q^*(x, a')$.*

A policy defined from a state-action value function as in the remark above is called a greedy policy. The implication of Remark 1 is that, if we want to compute an optimal policy, i.e., solve a Markov decision problem,⁶ we can alternatively compute the optimal state-action value function, from which an optimal policy is readily available. In our work, we focus on computing the optimal state-action value function q^* .

When a model of the environment is available to the agent, including the transition probability matrices and the expected reward function, planning methods such as policy iteration and value iteration can be used to solve the Markov decision problem. When such model is not available, reinforcement learning methods can be used instead.

2.2.1 Planning

In this section we assume the agent has knowledge of the transition probability matrices P_a , $a \in \mathcal{A}$, and of the expected reward function r . The goal of the agent is to solve the Markov decision problem, i.e., to compute q^* . We start by presenting the policy-iteration algorithm. Then, we move on to presenting the value-iteration algorithm. In both cases, we assume we are in a tabular setting, which means we store every state-action probability or value in a finite-sized matrix.

⁶We can also refer to the Markov decision problem as the control problem.

Policy iteration The policy-iteration algorithm iteratively improves a policy with respect to its value. Following the algorithm, the agent has an initial estimate of a policy, approximates its value, chooses the next policy as the greedy policy with respect to current value estimate and repeats until the policy does not change between iterations. The algorithm is guaranteed to converge after a finite number of policy improvement steps and the final policy is known to be optimal [47, Section 6.4]. In order to formalize the policy-iteration algorithm, we define the Bellman operator for prediction, H^π , mapping state-action value functions to state-action value functions as

$$(H^\pi q)(x, a) = \mathbb{E}[r_t + \gamma q(x_{t+1}, a_{t+1}) \mid x_t = x, a_t = a] \quad (9)$$

taking the expectation with respect to state, action, reward, next state and next action. As the operator is contractive, successively applying it to any initial state-action value function converges to q^π . The policy-iteration algorithm receives as input an initial policy π_0 and consists of a policy evaluation cycle, where the current policy is evaluated, inside a policy improvement cycle, where the current policy is improved. At time step t of the policy evaluation cycle, H^{π_t} is used to compute an approximation of the value of π_t , $\hat{q}^{\pi_t}$. Once such cycle converges, the algorithm takes a policy improvement step, where π_{t+1} is chosen as the greedy policy with respect to $\hat{q}^{\pi_t}$. It is guaranteed that for some T , $\pi_T = \pi_{T+1}$. The final policy π_T is optimal, with its value function q^{π_T} being equal to q^* .

Value iteration Contrarily to the policy-iteration algorithm, the value-iteration algorithm does not keep an explicit representation of a policy. It keeps only an estimate of the value of the policy. The algorithm follows a single cycle of iterations that converge to q^* . We define the Bellman operator H , mapping value functions to value functions as

$$(Hq)(x, a) = \mathbb{E} \left[r_t + \gamma \max_{a' \in \mathcal{A}} q(x_{t+1}, a') \mid x_t = x, a_t = a \right]. \quad (10)$$

taking the expectation with respect to the state, action, reward and next state. Successively applying the operator H to a value function composes the value-iteration algorithm. As H is contractive, the algorithm is guaranteed to converge to the optimal value function q^* .

In the following section we move on to reinforcement learning algorithms. Contrarily to the planning algorithms already presented, reinforcement learning algorithms require neither prior knowledge of the expected reward function nor of the transition probabilities.

2.3 Reinforcement learning

Without access to a model of its environment, a reinforcement learning agent learns to act by trial and error. We start by presenting reinforcement learning algorithms with tabular representations, and afterwards generalize the function approximation architecture.

2.3.1 Tabular representations

Despite the absence of a model of the transitions and reward of a Markov decision process, a reinforcement learning agent can learn in one of two ways. First, the agent can build estimates of such transition and rewards structures, and use the planning methods of the previous sections assuming those estimates are correct, leading to a class of methods that are called model-based. Alternatively, the agent can learn to act without explicit estimates of the transition and reward structures, leading to a class of methods called model-free. We recover two model-free reinforcement learning algorithms for the Markov decision problem.

SARSA SARSA is a reinforcement learning algorithm for learning the state-action value function of the policy the agent is using to interact with its environment. We know that such value function verifies the fixed-point equation (7), which can be written as

$$q^\pi(x, a) = (H^\pi q^\pi)(x, a)$$

If we rearrange the equation and write $H^\pi q^\pi$ explicitly, we obtain equivalently that

$$\mathbb{E} [r_t + \gamma q^\pi(x_{t+1}, a_{t+1}) - q^\pi(x_t, a_t) \mid x_t = x, a_t = a] = 0, \quad (11)$$

for all state-action pairs (x, a) . Equation (11) is on the form that motivates stochastic approximation algorithms, with variable q^π . We define the stochastic approximation update

$$q_{t+1}^\pi(x, a) = q_t^\pi(x, a) + \alpha_t (r_t + \gamma q_t^\pi(x_{t+1}, a_{t+1}) - q_t^\pi(x_t, a_t)),$$

forming the SARSA algorithm. Since SARSA learns the state-action value function of the policy the agent follows, it is called an on-policy algorithm. Even though SARSA appears an algorithm designed for prediction, i.e., to evaluate a certain given policy, we can use the SARSA algorithm to learn the optimal value function q^* . Specifically, as long as state-action pairs are visited infinitely often and the policy the agent follows is greedy in the limit [53], SARSA converges to q^* . Finally, we can say SARSA with increasingly greedy policies is the stochastic approximation counterpart to the policy iteration algorithm.

Q-learning Q-learning is a reinforcement learning algorithm that learns the optimal value function, regardless of the policy the agent uses to interact with the environment. For that reason, it is called an off-policy algorithm. We know that the optimal value function verifies, at all state-action pairs, the Bellman fixed-point equation (8), which we write:

$$q^*(x, a) = (Hq^*)(x, a)$$

We can rearrange the equation to obtain

$$\mathbb{E} \left[r_t + \gamma \max_{a' \in \mathcal{A}} q^*(x_{t+1}, a') - q^*(x_t, a_t) \mid x_t = x, a_t = a \right] = 0 \quad (12)$$

for all state-action pairs. We define Q-learning as the stochastic approximation update

$$q_{t+1}^*(x, a) = q_t^*(x, a) + \alpha_t \left(r_t + \gamma \max_{a' \in \mathcal{A}} q_t^*(x_{t+1}, a') - q_t^*(x_t, a_t) \right).$$

Q-learning converges to q^* as long as state-action pairs are visited infinitely often [59].

2.3.2 Function approximation

The two algorithms introduced above update the value of a single state-action pair at each time step. However, if the size of the state or the action space is sufficiently large, the learning process becomes prohibitively slow, or even impossible. The problem is known as the curse of dimensionality [7]. To learn in the presence of large state or action spaces, we stop assuming a tabular representation and use a function approximation architecture. Instead of learning each value individually, we learn the parameters of a function that outputs it. We denote the state random variable as x , the action random variable as a , the reward random variable as r , the next state random variable as x' and the next action random variable as

a' . We are assuming the distribution of these random variable is time-independent, which renders the setting closer to offline, or batch, reinforcement learning. This assumption makes the exposition and analysis simpler. We note, however, that the problems of convergence of Q -learning with function approximation are still present in this simpler setting. We will argue that they have deeper roots than the data. Consequently, we assume throughout the document the existence of a probability distribution over state-action pairs, μ . Let us also consider a differentiable function approximation space $\mathcal{Q} = \{q_{\mathbf{w}} : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}, \mathbf{w} \in \mathbb{R}^K\}$.

SARSA Suppose we wish to approximate q^π for some given policy π verifying the fixed point equation (7). We can define a loss function to optimize. Formally, we want to solve

$$\arg \min_{\mathbf{w} \in \mathbb{R}^K} \frac{1}{2} \mathbb{E}_\mu \left[((H^\pi q^\pi)(x, a) - q_{\mathbf{w}}(x, a))^2 \right],$$

We start by differentiating with respect to $\mathbf{w}$ and obtain

$$\mathbb{E}_\mu [(H^\pi q^\pi)(x, a) - q_{\mathbf{w}}(x, a)] \nabla q_{\mathbf{w}}(x, a) = 0.$$

Now, we assume that $(H^\pi q_{\mathbf{w}})(x, a) = \mathbb{E}[r + \gamma q_{\mathbf{w}}(x', a') \mid x = x, a = a]$ correctly estimates $(H^\pi q^\pi)(x, a)$, a technique we refer to as bootstrapping. We then obtain an equation that is of the form over which we can build a stochastic approximation algorithm:

$$\mathbb{E}_\mu [(r + \gamma q_{\mathbf{w}}(x', a') - q_{\mathbf{w}}(x, a)) \nabla q_{\mathbf{w}}(x, a)] = 0.$$

The SARSA algorithm with function approximation then becomes

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (r_t + \gamma q_{\mathbf{w}_t}(x'_t, a'_t) - q_{\mathbf{w}_t}(x_t, a_t)) \nabla q_{\mathbf{w}_t}(x_t, a_t). \quad (13)$$

Q-learning Suppose we wish to approximate q^* verifying the fixed point equation (8). We can define a loss function to optimize. Formally, we want to solve

$$\arg \min_{\mathbf{w} \in \mathbb{R}^K} \frac{1}{2} \mathbb{E}_\mu \left[((Hq^*)(x, a) - q_{\mathbf{w}}(x, a))^2 \right].$$

We start by differentiating with respect to $\mathbf{w}$ and obtain

$$\mathbb{E}_\mu [(Hq^*)(x, a) - q_{\mathbf{w}}(x, a)] \nabla q_{\mathbf{w}}(x, a) = 0.$$

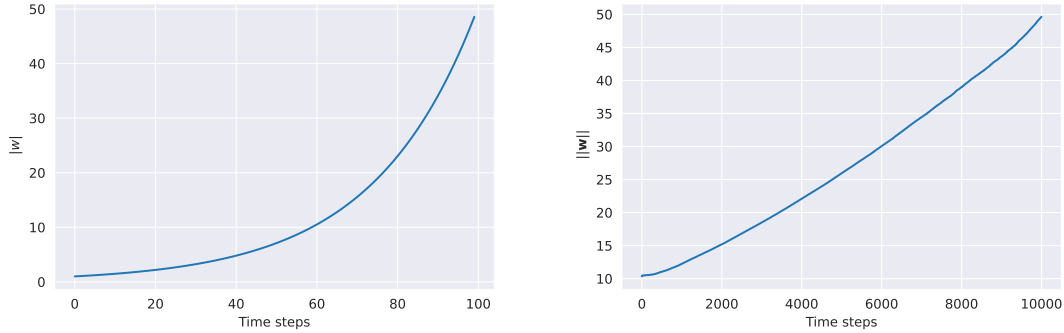
As above, we assume that $(Hq_{\mathbf{w}})(x, a) = \mathbb{E}[r + \gamma q_{\mathbf{w}}(x', a') \mid x = x, a = a]$ correctly estimates $(Hq^*)(x, a)$. We then obtain an equation that is of the form over which we can build a stochastic approximation algorithm:

$$\mathbb{E}_\mu [(r + \gamma q_{\mathbf{w}}(x', a') - q_{\mathbf{w}}(x, a)) \nabla q_{\mathbf{w}}(x, a)] = 0.$$

The Q -learning algorithm with function approximation becomes

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \left(r_t + \gamma \max_{a' \in \mathcal{A}} q_{\mathbf{w}_t}(x'_t, a') - q_{\mathbf{w}_t}(x_t, a_t) \right) \nabla q_{\mathbf{w}_t}(x_t, a_t).$$

While natural, the introduction of function approximation in reinforcement learning has major drawbacks. We do know that tabular SARSA and Q -learning are guaranteed to converge. However, the same guarantees do not hold in the presence of function approximation, even if the architecture is as seemingly harmless as a linear one. Divergence happens in well-known examples where the parameters of the approximation do not approach a solution, either oscillating within a window [10, 19] or growing without bound [4, 57].



(a) Results in the $w \rightarrow 2w$ counter example.

(b) Results in Baird's counter example.

Figure 2.1: Divergence of the parameters in the $w \rightarrow 2w$ and Baird's counter examples. As the learning process evolves, the norms of the parameters of the function approximation architectures diverge. As the processes evolve, the norm of the parameters grows without bound, getting further from the desired solution, zero.

2.4 Divergence of Q -learning

In this section, we showcase the divergence of Q -learning in well-known practical examples from the literature and then explore its underpinnings from a theoretical perspective.

2.4.1 Examples of undesirable behavior with linear function approximation

We show two examples of divergence of Q -learning with linear function approximation in small Markov decision processes. Both counter examples presented are contrived and were specifically designed to bring Q -learning to failure. But at the same time, they show that even in simple Markov decision processes, using simple function approximation architectures, Q -learning can fail to provide any solution, let alone an optimal one. We can hypothesize that similar phenomena may happen in certain subsets of states and actions in bigger Markov decision processes and with more general approximation architectures. Finally, we showcase an example of divergence of SARSA, supporting the argument that the convergence problem of Q -learning is more profound than the off-policy updates.

$w \rightarrow 2w$ counter example We refer to the first divergent example as $w \rightarrow 2w$. The example is due to Tsitsiklis and Van Roy [57]. It comprises a Markov decision process with two states, which we call states s_1 and s_2 , a single action, and a reward function that is constantly zero. From state s_1 , the agent always transitions to state s_2 , from state s_2 the agent remains in state s_2 . Since all rewards are zero, the value function is zero on both states. The function approximation architecture used is linear and parameterized by a real-valued w . The feature is $\phi : \mathcal{X} \rightarrow \mathbb{R}$ such that $\phi(s_1) = 1$ and $\phi(s_2) = 2$. The value function is correctly approximated if $w = 0$. Figure 2.2 depicts the process and the features. The example is simple in the sense that the state space is small, the action space is trivial and the function approximation architecture allows for correct representation of the value function. If we set a discount factor $\gamma = 0.9$, a learning rate $\alpha_t = 0.1$, an initial $w_0 = 1$, sample states according to the uniform distribution, use single step transitions and use Q -learning's update rule,⁷ the parameter w diverges. Specifically, w grows without bound.

⁷Since the action space is trivial, Q -learning's update is the same as SARSA's.



Figure 2.2: Transition diagram of the process in the $w \rightarrow 2w$ counter example.

Figure 2.1a shows how the absolute value of w evolves along the process.

Baird’s counter example We refer to the second divergent example as Baird’s counter example [4], even though it was subsequently slightly modified by Sutton and Barto [54]. The Markov decision process has seven states and two actions, which we call b_1 and b_2 . Action b_1 transitions the agent to any of the first six states uniformly, action b_2 transitions the agent to the seventh state. All rewards are zero and so are the Q -values. For the first six states, we define state-features $\psi : \mathcal{X} \rightarrow \mathbb{R}^8$ such that $\psi(x)$ is 2 in the x -th component, 1 in the eight component and 0 otherwise. For the seventh state, ψ is 1 in the seventh component, 2 in the eight component, and 0 otherwise. Then, the state-action features are $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^{16}$ such that $\phi(x, b_1) = [\psi^T(x), 0, \dots, 0]^T$ and $\phi(x, b_2) = [0, \dots, 0, \psi^T(x)]^T$. Figure 2.3 depicts the process and the features. As in the previous counter example, if

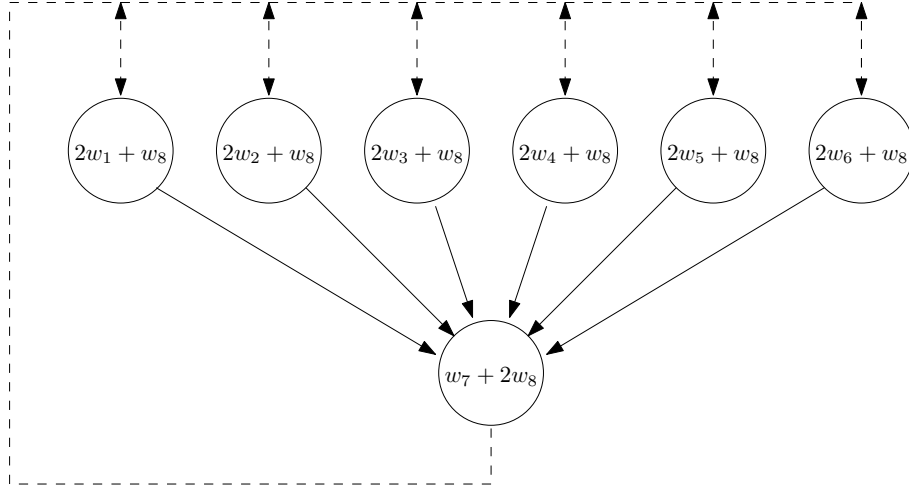


Figure 2.3: Transition diagram of the process in Baird’s counter example.

the parameters are $\mathbf{w} = \mathbf{0}$, the correct state-action value function is represented. We set the discount factor $\gamma = 0.99$, the learning rate $\alpha_t = 0.01$ and the initial parameters are $\mathbf{w}_0 = [1, 1, 1, 1, 1, 1, 10, 1, 1, 1, 1, 1, 1, 1, 1]$. The behavioral policy chooses action b_1 with probability $\frac{6}{7}$ and action b_2 with probability $\frac{1}{7}$. Therefore, the next state distribution is uniform and every state is visited equally and infinitely often. Even though the Markov decision process and the linear representation are benign, in the sense that the correct state-action value function is representable and the features are linearly independent, the norm of the parameter vector diverges to infinity. Again, the parameters grow without bound. Figure 2.1b shows the norm of the vector $\mathbf{w}$.

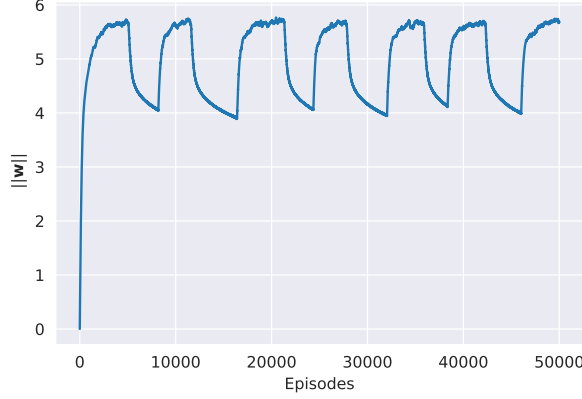


Figure 2.4: Divergence of the parameters in the chattering counter example. As the learning process evolves, the norms of the parameters of the function approximation architectures diverge, oscillating within a window without converging. As the processes evolve, the norm of the parameters oscillates periodically and fails to converge.

Chattering counter example We refer to the final example as the chattering counter example. The example is due to Gordon [19] and showcases the divergence of SARSA with linear function approximation. It comprises a Markov decision process with four states and two actions. From the initial state s_I , the agent can either move to an upper state s_U , through action b_U or to a lower state s_L , through action b_L . From these, the agent moves to a final state s_F . The action of moving up from the initial state has a reward of zero, the action of moving down from the initial state has a reward of zero, moving down to the final state has a reward of two, moving up to the final state has a reward of 1. The function approximation architecture used is linear and parameterized by $\mathbf{w} \in \mathbb{R}^4$. The features are $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$ and such that for the initial state $\phi(s_I, b_U) = [1, 0, 0, 0]^T$ and $\phi(s_I, b_L) = [0, 1, 0, 0]^T$, for the final state $\phi(s_F, \cdot) = [0, 0, 0, 1]$, regardless of the action, and for the upper and lower states, $\phi(\cdot, \cdot) = [0, 0, 1, 0]$, regardless of the action. Figure 2.5 depicts the process and the features. For the experiments, we consider a discount factor

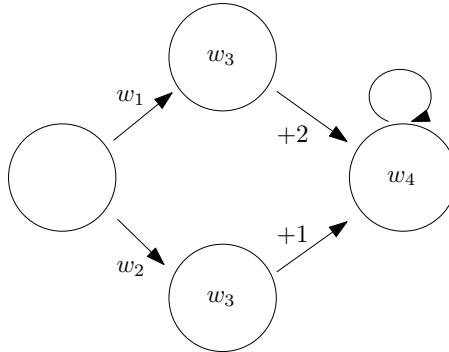


Figure 2.5: Transition diagram of the process in the chattering counter example.

$\gamma = 11$, a learning rate $\alpha_t = 0.01$ and an ϵ -greedy policy with $\epsilon = 0.1$. If the initial parameters are $\mathbf{w} = \mathbf{0}$, they do not converge. Lowering the discount factor, learning rate or ϵ does not result in convergence either [19]. Figure 2.4 shows the norm of the vector $\mathbf{w}$.

2.4.2 Discussion

Here, we expand on reasons for the divergence of Q -learning. We start by analyzing the $w \rightarrow 2w$ example once more. In this example, as there is only a single action, the updates of SARSA are the same as of Q -learning. Consequently, the arguments apply to both.

$w \rightarrow 2w$ counter example In this example, Q -learning is driven by the o.d.e.

$$\dot{w} = \mathbb{E} [\phi(x) (r(x) + \gamma q_w(x') - q_w(x))] , \quad (14)$$

where the expectation is taken with respect to the states x , which can be s_1 or s_2 with equal probability. The next state x' is always s_2 and the reward $r(x)$ is always zero. Since $\phi(x)$ equals 1 if $x = s_1$ and equals 2 if $x = s_2$ and $q_w(x) = \phi(x)w$, the o.d.e. (14) simplifies:

$$\dot{w} = \left(3\gamma - \frac{5}{2}\right) w. \quad (15)$$

Zero is an equilibrium of o.d.e. (15) but it is not stable if $\gamma > \frac{5}{6}$. Therefore, even if the dynamical system starts close to the solution, it does not converge to it and neither does the discretized stochastic variant of the o.d.e. that is Q -learning.

Linear function approximation Generalizing the analysis, but still considering linear function approximation, an equilibrium for Q -learning verifies

$$\mathbb{E}_\mu \left[\phi(x, a) \left(r(x, a) + \gamma \max_{a' \in \mathcal{A}} q_w(x', a') - q_w(x, a) \right) \right] = 0,$$

which states, equivalently, that

$$\mathbb{E}_\mu [\phi(x, a) q_w(x, a)] = \mathbb{E}_\mu \left[\phi(x, a) \left(r(x, a) + \gamma \max_{a' \in \mathcal{A}} q_w(x', a') \right) \right] = 0.$$

Using that in the linear function approximation case $q_w(x, a) = \phi^T(x, a)\mathbf{w}$ and denoting $\Sigma = \mathbb{E}_\mu [\phi(x, a)\phi^T(x, a)]$, we obtain

$$\Sigma \mathbf{w} = \mathbb{E}_\mu \left[\phi(x, a) \left(r(x, a) + \gamma \max_{a' \in \mathcal{A}} q_w(x', a') \right) \right],$$

assuming Σ is invertible and, finally,

$$\mathbf{w} = \Sigma^{-1} \mathbb{E}_\mu \left[\phi(x, a) \left(r(x, a) + \gamma \max_{a' \in \mathcal{A}} q_w(x', a') \right) \right]. \quad (16)$$

If the right-hand side of the fixed-point equation (16) is contractive, there exists a unique solution and the dynamical system converges. Unfortunately, as can be seen in the simple $w \rightarrow 2w$ example, in general, that is not the case and Q -learning can diverge. In the example we saw that, when $\gamma > \frac{5}{6}$, a solution to Equation (16) exists but, since the right-hand side is not contractive, Q -learning does not converge to the solution. We can also build an example where the solution to Equation (16) does not even exist. For that, we consider the Markov decision process with the same state, action space and dynamics but where the reward r is strictly positive. An equilibrium for the o.d.e. (14) must satisfy the equation

$$3r + (6\gamma - 5)w = 0.$$

The equation does not have a solution when $\gamma = \frac{5}{6}$. The argument also applies to SARSA.

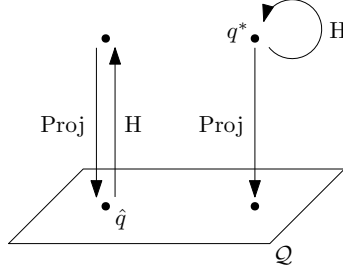


Figure 2.6: Representation of relevant functions and operators. The optimal value function is q^* , and it is the fixed point of the Bellman operator H . The solution of Q -learning is $\hat{q}$, the fixed point of the composition of H and the projection operator Proj onto the function approximation space $\mathcal{Q}$. However, $\hat{q}$ may not exist. Furthermore, $\hat{q}$ does not equal $\text{Proj } q^*$.

General function approximation Generalizing once more, and considering unrestricted function approximation architectures, from a functional perspective, Q -learning is a stochastic approximation algorithm for finding $\hat{q}$ that solves the equation

$$q = \text{Proj}(Hq), \quad (17)$$

where Proj is the projection operator mapping state-action value functions to the function approximation space $\mathcal{Q}$, formally defined as

$$\text{Proj } q = \arg \min_{w \in \mathbb{R}^K} \frac{1}{2} \mathbb{E}_\mu \left[(q(x, a) - q_w(x, a))^2 \right].$$

Since there is no guarantee that the composed operator $\text{Proj } H$ is contractive, Equation (17) may not have a solution, i.e., $\hat{q}$ may not exist. When $\hat{q}$ does exist, there is still no guarantee that successively applying $\text{Proj } H$ converges to it. Figure 2.6 graphically depicts the considerations made. For a fixed policy π , the fixed-point of SARSA verifies

$$q = \text{Proj}(H^\pi q). \quad (18)$$

As we saw before, there is no guarantee that such fixed-point exists. Furthermore, as π become increasingly greedy, H^π tends to H . By continuity, one solution for Equation (17) must in that case be the same as for Equation (18). If Equation (17) does not have a fixed-point, neither does Equation (18) when π is greedy with respect to q . As a consequence, same as Q -learning, SARSA is not guaranteed to converge to a solution: even though it is guaranteed to converge to a region [19], the updates may oscillate within the region, as seen in the chattering counter example. Following this reasoning, we mostly dedicate our efforts to the problem of approximating q^* by focusing on Q -learning.

3 Related work

In this section we critically analyze work that is related to our own. We focus the section on works that study the convergence of Q -learning methods under different settings. Specifically, we review what is known regarding the asymptotic behavior of Q -learning in the three settings identified as objects of this thesis in Section 1, although in a different order. First, we discuss results on the convergence of Q -learning with linear function approximation. Afterwards, we discuss results on the convergence of Q -learning with non-linear function approximation. Then, we discuss results regarding full-gradient descent methods in reinforcement learning. In the latter, we also consider non-linear function approximation. At that point, we discussed the class of policy-based methods, opposing it to Q -learning methods. Finally, we discuss results on Q -learning in non-stationary settings.

3.1 Linear function approximation

We start by analyzing the works that provide results on the convergence of Q -learning with non-trivial linear function approximation, i.e., when the function approximation architecture is the linear span of a set of features that are not tabular representations of the state-action space. First, a few works provide conditions under which the standard Q -learning method, that performs semi-gradient descent on the Bellman error, converges. Others, propose a different learning objective and thus result in significantly modified variants of Q -learning. Finally, other works exploit the finding that regularization can ensure convergence of the Q -learning updates. Respectively, we expand on each of the three directions below.

3.1.1 Convergence conditions

Around the time the divergence of Q -learning was established, through the counter-examples presented in Section 2.4, some works aimed at understanding specific conditions under which Q -learning would provably converge with linear function approximation. The first works proposed restrictions of the function approximation architectures themselves. They establish that with some specific choices of features, Q -learning is guaranteed to converge [52, 44, 56]. The linear architectures considered across the three works are soft extensions of what would be the one-hot representation in the tabular case. Afterwards, another work considered again general linear function approximation architectures. Melo et al. [38] prove that Q -learning with linear function approximation converges if the distribution of state-action pairs the agent uses to learn is sufficiently close to the distribution that the optimal policy would induce. Such case renders Q -learning closer to its on-policy variant, SARSA. As a consequence, the work establishes the convergence of SARSA, the on-policy version of Q -learning, with linear function approximation. Conversely, it implies that, if the discount factor is small enough, the convergence of Q -learning also follows.

3.1.2 Gradient-TD methods

Instead of finding conditions under which Q -learning converges, a different line of works proposes to take a step back and modify the objective that Q -learning is trying to solve. The solution Q -learning may find must verify $q = \text{Proj}(\text{H}q)$ or, equivalently,

$$\text{Proj}(\text{H}q) - q = 0.$$

The solution may not exist and the approach may result in divergence as already mentioned. Maei et al. [33] propose to perform full-gradient descent on a different objective, called the projected Bellman error. The motivation is that, even though $\text{Proj}(\text{H}q) - q$ may never be zero, we can aim at minimizing the distance between $\text{Proj}(\text{H}q_w)$ and q_w in the function approximation space. The resulting algorithm, called greedy-GQ, is part of a class of gradient-TD methods. Formalizing the objective, gradient-TD methods aim at finding the parameter vector $\mathbf{w}^*$ such that

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^K} \frac{1}{2} \mathbb{E}_{\mu} \left[((\text{Proj}(\text{H}q_w))(\mathbf{x}, \mathbf{a}) - q_w(\mathbf{x}, \mathbf{a}))^2 \right].$$

Being a full gradient method, greedy-GQ is provably convergent in the linear state-action value function space. However, the method can converge to strictly local minima and there is no guarantee that the resulting greedy policy is a good control policy [50]. Gradient-TD methods are also less efficient than semi-gradient methods [16, 34].

3.1.3 Regularized methods

The problem of divergence of Q -learning with function approximation was significantly revived after the empirical success of Q -learning with deep neural networks [40]. One of the components of the renowned deep Q -network (DQN) is a target network that aims at compensating the instability generated by the semi-gradient updates. While a target network does not make Q -learning convergent even in the linear function approximation case, its empirical success inspired novel theoretical results. Our work [12], discussed in Section 4.1, and the works of Zhang et al. [63] and Chen et al. [13], provided convergence guarantees for Q -learning with target networks. Additionally, another work points out that the target network can be seen as a form of regularization [46]. As hinted by Farahmand [18], several works are proving that several forms of regularization of the Q -values or the parameters themselves can stabilize Q -learning, resulting in convergent algorithms [1, 30, 63]. However, the introduction of any of the regularizers biases the limit solution encountered by Q -learning in ways that are not fully understood. Specifically, it is unclear how good the greedy policies resulting from the learned value functions are.

3.2 Non-linear function approximation

As discussed, the behaviour of Q -learning with linear function approximation has been the focus of several theoretical works. In practice, however, Q -learning is mostly used with non-linear function approximation architectures, especially neural ones [40]. Still, there are works that address this more general setting. A recent work suggests a loss function that is decreasing over time, assuming the neural network converges to the targets from a target network at each step [58]. However, having a loss function that is monotonically decreasing does not imply that the parameters of the approximator or the Q -values are converging. Cai et al. [11] and Xu and Gu [60] provide finite-time analysis of Q -learning with over-parameterized neural networks that imply that as the size of the network grows to infinity, convergence is guaranteed at a sub-linear rate. While being an interesting result, as the size of the network grows to infinity, the learning architecture also grows closer to a tabular representation. However, the results do not imply convergence when a finite-sized network is employed, deeming them somehow detached from practice.

3.3 Policy-based methods

The class of policy-based methods, directly update the parameters of a policy on a function space Π . Policy-based methods, include some of the state-of-the-art algorithms in control problems of reinforcement learning. Examples of those are actor-critic methods [5, 25]. Actor-critic methods also update on the parameters of a value function in $\mathcal{Q}$, the critic, that evaluates the policy, the actor. The actor is updated through stochastic gradient ascent on the critic, and relying on the policy-gradient theorem [55]. Letting

$$J(\pi) = \mathbb{E}_{x \sim \mu} [q^\pi(x, a)], \quad (19)$$

where actions are taken according to π , the theorem states that

$$\nabla J(\pi) = \mathbb{E} [((\nabla \log \pi)(x, a)) (q^\pi(x, a) - b(x))], \quad (20)$$

where b is a baseline that does not bias the gradient estimation but can reduce or increase the variance. PPO [51] is one such actor-critic methods, where the baseline used is the state value function $b(x) = \mathbb{E}[q^\pi(x, a)]$, added of the optimisation of a surrogate objective loss. Other well-known actor-critic methods are A3C [41], its synchronous version A2C, DDPG [29], IMPALA [17] and SAC [20]. In this thesis, we focus on Q -learning methods. We discuss possible limitations of policy-based methods in light of our research problem, the one of approximating the optimal value-function q^* , which indirectly results in approximating an optimal policy, even though the converse is not true.

Computation Policy-based methods increase on the computational complexity of Q -learning methods. Most policy-based methods, including actor-critic methods, require storing the parameters of the policy. These parameters are stored in addition to the parameters of the value function. Contrarily, Q -learning methods require only the latter, making them more memory efficient in this sense. The policy parameters also require gradient ascent updates. Since the policy space is in principle non-linear, these updates can be expensive. The computation cost adds to the cost of updating only the value function.

Local minima In general, J maps parameterized policies π to $\mathbb{R}$ in a non-convex way. Therefore, convergence of the policies happens to a policy whose value is only a local minima in the value function space $\mathcal{Q}$. Interestingly, departing from the classic linear programming formulation of the Markov decision problem [36] and considering a logistic Bellman error instead of the squared Bellman error, Bas-Serrano et al. [6] arrive at an algorithm that converges to the optimal policy. The convergence result therein assumes, for example, that q^* is in $\mathcal{Q}$ and that the Markov decision process is linearly factored. In our studies, we aim at considering extremely relaxed assumptions on the data distribution μ , the approximation space $\mathcal{Q}$ and the Markov decision process.

Policy evaluation The updates on the policy require, in principle, its evaluation. As seen in Section 2.4, approximating the value of a policy in a function approximation space lacks convergence guarantees and divergence can happen. We expect that, while focusing on Q -learning methods, the contributions of this thesis also address the policy evaluation problem with function approximation. Alternatively to approximating the value using the Bellman operator, we can perform rollouts of the policy on the Markov decision process and consider the collected discounted reward. The drawbacks of the approach are the assumption that we can simulate the environment without restrictions, added computation time in simulating the environment, and much increased variance.

3.4 Learning with non-stationarities

There is also work on the asymptotic behaviour of reinforcement learning on non-stationary Markov decision processes [21, 22, 28, 35]. The results focus on a class of history-dependent Markov decision processes such that the optimal Q -values are independent of the history of the process throughout the interaction, which is not generally the case if the environment is non-stationary. It is unknown in which other types of non-stationarities, for instance ones that result from concurring simultaneous learning processes, Q -learning will converge, including in the tabular case. Specifically, we do not know if Q -learning converges in hierarchical settings [15], changing state or action spaces, discount factors, transition dynamics or reward structures. We can also consider changes in the data along the learning process as a non-stationarity.

3.5 Full-gradient methods

Focused on the linear function approximation case but with impacts on the general case, an early work [4] had called attention for the fact that Q -learning does not perform full-gradient descent on the Bellman error, i.e., does not attempt to find the parameters $\mathbf{w}^*$ such that

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^K} \frac{1}{2} \mathbb{E}_{\mu} \left[((Hq_{\mathbf{w}})(\mathbf{x}, \mathbf{a}) - q_{\mathbf{w}}(\mathbf{x}, \mathbf{a}))^2 \right].$$

Full-gradient Q -learning would be a convergent method. However, convergence would be slower. Additionally, it demands the possibility to perform two unbiased samples from each state throughout the interaction [4] if the state transitions on the environment are not deterministic. The last requirement is usually referred to as the double sampling issue [54, Chapter 11]. For the case of non-linear function approximation, Avrachenkov et al. [3] prove that full-gradient descent on the Bellman error results in a convergent Q -learning scheme. However, the solution may be minimizing the Bellman error only locally, and the resulting control policy may not be good, as discussed by Sutton and Barto [54].

Other works investigate, in practice, the quality of the solutions obtained by full-gradient and semi-gradient methods [49, 50, 61, 62]. The insights we obtain from such works are various. Saleh and Jiang [49] show that, in deterministic environments, the solutions of full-gradient methods may often be good. They highlight, however, that when the data deviates much from the on-policy distribution, full-gradient Q -learning can be severely harmed. Scherrer [50] shows that, in general, full-gradient methods perform worse more often than semi-gradient methods but that, when they fail, it is not by as much as semi-gradient methods do. Yin et al. [61] could not obtain good solutions from full-gradient descent methods. Finally, Zhang et al. [62] shows that, for policy evaluation, a certain convex combination of the full-gradient and semi-gradient update results in increased performance in comparison with the extreme cases, of either full-gradient or semi-gradient updates.

4 Linear function approximation

As seen in Section 2.4, Q -learning may fail to converge when used with linear function approximation. In this section, we provide alternative methods to Q -learning and its on-policy relative, SARSA, with linear function approximation, that are guaranteed to converge with probability 1. The proposed methods are two time-scale variations inspired by, respectively, the algorithms of deep Q -networks [40] and approximate policy iteration [45]. In this section, we consider linear function approximation spaces. A linear function approximation space is generated by features $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^K$, so that $q_{\mathbf{w}}(x, a) = \phi^T(x, a)\mathbf{w}$.

4.1 Q -learning

We introduce an off-policy two time-scale stochastic approximation algorithm called CQL, for coupled Q -learning. Our proposed CQL algorithm takes inspiration from the two critical techniques that were used for the DQN algorithm: the target network and experience replay. A target network is a delayed copy of the main network, updated only every fixed number of steps, in order to mitigate the negative impact of bootstrapping; experience replay works by keeping a buffer of transitions $\mathcal{D}$ from where samples are drawn and used to perform the updates. The replay buffer makes the data less correlated than it is if we consider the fully Markovian setting and slows down the data distribution shift that arises from having a changing behavior policy. Just like in DQN, CQL also encompasses a target network, although in a modified way. In our case, instead of copying the target network every fixed number of steps, we update it in every step, but slowly. Also, instead of simply copying the main network, we update the target network in order to approximate a regularized version of the main network. Our data setting resembles the one of experience replay, where the agent has access to a fixed batch of samples. The CQL algorithm performs the following coupled updates to the main parameters $\mathbf{v}_t \in \mathbb{R}^K$ and the target parameters $\mathbf{u}_t \in \mathbb{R}^K$:

$$\begin{aligned}\mathbf{v}_{t+1} &= \mathbf{v}_t + \alpha_t \left(r_t + \gamma \max_{a' \in \mathcal{A}} q_{\mathbf{u}_t}(x'_t, a') - q_{\mathbf{v}_t}(x_t, a_t) \right) \phi(x_t, a_t), \\ \mathbf{u}_{t+1} &= \mathbf{u}_t + \beta_t (\phi(x_t, a_t) \phi^T(x_t, a_t) \mathbf{v}_t - \mathbf{u}_t).\end{aligned}$$

We can show the proposed scheme converges with probability 1 using the general convergence result for two time-scale stochastic approximation reproduced in Section 2.1.1.

Assumption 1. *We assume the following hold:*

1. (x_t, a_t, x'_t, r_t) is sampled from $\mathcal{D} = \{(x_i, a_i, x'_i, r_i), i \in \mathbb{N}_0\}$ with distribution μ .
2. The matrix $\Sigma = \mathbb{E}_{\mu} [\phi(x, a) \phi^T(x, a)]$ is non-singular and $\|\phi(x, a)\|_2 \leq 1$, for all (x, a) .

Theorem 2. *For any initial parameters $\mathbf{v}_0$ and $\mathbf{u}_0$, it holds that*

$$(\mathbf{v}_t, \mathbf{u}_t) \rightarrow (\mathbf{v}^*(\mathbf{u}^*), \mathbf{u}^*),$$

where $\mathbf{u}^*$ is the unique solution to

$$\mathbf{u}^* = \mathbb{E} \left[\left(r(x, a) + \gamma \max_{a' \in \mathcal{A}} \phi^T(x', a') \mathbf{u}^* \right) \phi(x, a) \right] \quad (21)$$

and $\mathbf{v}^* : \mathbb{R}^K \rightarrow \mathbb{R}^K$ is such that $\mathbf{v}^*(\mathbf{u}) = \Sigma^{-1} \mathbf{u}$.

Proof. We present only the main steps of the proof. We establish the result by directly applying the two time-scale stochastic approximation result reproduced in Section 2.1.1, Theorem 1. To do such application, it amounts to show that our algorithm satisfies each condition of the theorem. We start by defining the mean fields $f, g : \mathbb{R}^K \times \mathbb{R}^K \rightarrow \mathbb{R}^K$ as

$$\begin{aligned} f(\mathbf{v}_t, \mathbf{u}_t) &= \mathbb{E}_\mu \left[\phi(\mathbf{x}_t, \mathbf{a}_t) \left(r_t + \gamma \max_{a' \in \mathcal{A}} q_{\mathbf{u}_t}(\mathbf{x}'_t, a') - q_{\mathbf{v}_t}(\mathbf{x}_t, \mathbf{a}_t) \right) \right], \\ g(\mathbf{v}_t, \mathbf{u}_t) &= \mathbb{E}_\mu [\phi(\mathbf{x}_t, \mathbf{a}_t) \phi^T(\mathbf{x}_t, \mathbf{a}_t)] \mathbf{v}_t - \mathbf{u}_t. \end{aligned}$$

We also define martingale difference sequences of noise $\{\mathbf{m}_t\}_{t \in \mathbb{N}}$ and $\{\mathbf{n}_t\}_{t \in \mathbb{N}}$ as

$$\begin{aligned} \mathbf{m}_{t+1} &= \phi(x_t, a_t) \left(r_t + \gamma \max_{a' \in \mathcal{A}} q_{\mathbf{u}_t}(\mathbf{x}'_t, a') - q_{\mathbf{v}_t}(x_t, a_t) \right) - g(\mathbf{u}_t, \mathbf{v}_t) \\ \mathbf{n}_{t+1} &= (\phi(x_t, a_t) \phi^T(\mathbf{x}_t, \mathbf{a}_t) \mathbf{v}_t - \mathbf{u}_t) - f(\mathbf{u}_t, \mathbf{v}_t). \end{aligned}$$

Both f and g are Lipschitz-continuous. Letting $\mathcal{F}_t = \sigma(\{(\mathbf{u}_\tau, \mathbf{v}_\tau, \mathbf{m}_\tau, \mathbf{n}_\tau), \tau = 0, \dots, t\})$, we can show that

$$\begin{aligned} \mathbb{E} [\|\mathbf{m}_{t+1}\|^2 \mid \mathcal{F}_t] &\leq c_{\mathbf{m}}(1 + \|\mathbf{u}_t\|^2 + \|\mathbf{v}_t\|^2), \\ \mathbb{E} [\|\mathbf{n}_{t+1}\|^2 \mid \mathcal{F}_t] &\leq c_{\mathbf{n}}(1 + \|\mathbf{u}_t\|^2 + \|\mathbf{v}_t\|^2), \end{aligned}$$

for some constants $c_{\mathbf{m}}, c_{\mathbf{n}} > 0$. In analyzing two time-scale algorithms we follow the standard notion that, since $\beta_t = o(\alpha_t)$, the updates to $\mathbf{v}_t$ proceeds at a “faster” timescale than those to $\mathbf{u}_t$. Thus, when viewed from the faster time-scale, $\mathbf{u}_t$ appears to be static. The update for $\mathbf{u}_t$ takes the general form $\mathbf{u}_{t+1} = \mathbf{u}_t + \alpha_t \Delta \mathbf{u}_t$, where

$$\Delta \mathbf{u}_t = \frac{\beta_t}{\alpha_t} (\phi(x_t, a_t) \phi^T(x_t, a_t) \mathbf{v}_t - \mathbf{u}_t) \rightarrow 0$$

as long as $\mathbf{v}_t$ and $\mathbf{u}_t$ remain bounded. With this in mind, for a fixed $\mathbf{u}$, we have the o.d.e.

$$\dot{\mathbf{v}}_t = g(\mathbf{v}_t, \mathbf{u}),$$

which has a unique globally asymptotically stable equilibrium

$$\mathbf{v}^*(\mathbf{u}) = \Sigma^{-1} \mathbb{E} \left[\phi(\mathbf{x}_t, \mathbf{a}_t) \left(r_t + \gamma \max_{a' \in \mathcal{A}} \phi^T(\mathbf{x}'_t, a') \mathbf{u} \right) \right].$$

The global asymptotic stability of $\mathbf{v}^*(\mathbf{u})$ can be established by a Lyapunov argument, using the Lyapunov function $l_{\mathbf{u}}(\mathbf{v}) = \frac{1}{2} \|\mathbf{v} - \mathbf{v}^*(\mathbf{u})\|^2$. Additionally, $\mathbf{v}^*$ can also be shown to be Lipschitz-continuous. Finally, defining

$$g_\infty(\mathbf{v}) = \lim_{c \rightarrow \infty} \frac{g(c\mathbf{v}, \mathbf{u})}{c} = -\Sigma \mathbf{v},$$

we can show that the origin is globally asymptotically stable for the o.d.e. $\dot{\mathbf{v}}_t = g_\infty(\mathbf{v}_t)$. From Bhatnagar et al. [8], we conclude that $\sup_t \|\mathbf{v}_t\| < \infty$ w.p. 1. Conversely, when viewed from the slower time scale, $\mathbf{v}_t$ is in equilibrium. We have the o.d.e. $\dot{\mathbf{u}}_t = f(\mathbf{v}^*(\mathbf{u}_t), \mathbf{u}_t)$, which also has a unique globally asymptotically stable equilibrium

$$\mathbf{u}^* = \mathbb{E} \left[\phi(\mathbf{x}_t, \mathbf{a}_t) \left(r_t + \gamma \max_{a' \in \mathcal{A}} \phi^T(\mathbf{x}'_t, a') \mathbf{u}^* \right) \right]. \quad (22)$$

The existence of the fixed point in (22) can be established from the Banach fixed-point theorem, since the right-hand side is a contraction in $\mathbf{u}$; that $\mathbf{u}^*$ is globally asymptotically stable can again be established by a Lyapunov argument, using the function $l(\mathbf{u}) = \frac{1}{2}\|\mathbf{u} - \mathbf{u}^*\|^2$. Finally, we define

$$f_\infty(\mathbf{u}) = \lim_{c \rightarrow \infty} \frac{f(\mathbf{v}^*(c\mathbf{u}), c\mathbf{u})}{c} = \mathbb{E} \left[\phi(\mathbf{x}_t, \mathbf{a}_t) \gamma \max_{a' \in \mathcal{A}} \phi^T(\mathbf{x}_t, a') \mathbf{u} \right] - \mathbf{u}.$$

We can show that the origin is an asymptotically stable equilibrium for the o.d.e. $\dot{\mathbf{u}}_t = f_\infty(\mathbf{u}_t)$, from where we repeat our previous argument to conclude that $\sup_t \|\mathbf{u}_t\| < \infty$ w.p. 1. Since all conditions have been verified, the conclusion follows. $\square$

4.2 Approximate policy iteration

The CQL algorithm introduced in the previous section is an off-policy algorithm. It does not require that the policy used to interact with the environment to be constantly changing in the direction of the state-action value function. Now, we introduce an on-policy method that we call 2TAP for two time-scale approximate policy iteration. Our proposed algorithm takes inspiration from the original approximate policy iteration algorithm [45]. In approximate policy iteration, a SARSA cycle, as defined in Section 2.3.2 is called inside a policy improvement step, where a new policy is greedy with respect to the state-action value estimates. The algorithm thus adapts the policy iteration planning algorithm, as defined in Section 2.2.1 to the reinforcement learning setting with function approximation. In 2TAP, along a faster time scale, we use a regularized SARSA update to predict the value of a policy which is being updated along a slower time-scale. Our parameterized policy, given a state, outputs a Boltzmann-like distribution over the actions. We update the policy parameters in the direction of the parameters of the value function. We consider a projection $\bar{\cdot} : \mathbb{R}^K \rightarrow \mathcal{B}_\rho$, $\mathcal{B}_\rho = \{z \in \mathbb{R}^k, \|z\|_2 \leq \rho\}$, defined as $\bar{z} = \arg \min_{\bar{z} \in \mathcal{B}_\rho} \|z - \bar{z}\|_2$ to ensure the value parameters are bounded. The 2TAP algorithm performs the following coupled updates to the value parameters $\mathbf{w}_t \in \mathbb{R}^K$ and the policy parameters $\theta_t \in \mathbb{R}^K$:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \left(\overline{(r_t + \gamma q_{\mathbf{w}_t}(\mathbf{x}'_t, \mathbf{a}'_t)) \phi(\mathbf{x}_t, \mathbf{a}_t)} - \mathbf{w}_t \right) \quad (23)$$

$$\theta_{t+1} = \theta_t + \beta_t (\mathbf{w}_t - \theta_t). \quad (24)$$

The policy space is $\Pi = \{\pi_\theta : \theta \in \mathbb{R}^K\}$, such that a policy π_θ in Π is such that

$$\pi_\theta(a | x) = \epsilon + (1 - |\mathcal{A}|\epsilon) \frac{\exp(\eta \phi^T(x, a) \theta)}{\sum_{a' \in \mathcal{A}} \exp(\eta \phi^T(x, a') \theta)},$$

where $\epsilon \in]0, \frac{1}{|\mathcal{A}|}[$ and $\eta > 0$ is a temperature parameter, resembling the space considered by Bas-Serrano et al. [6]. By defining the policies this way, we ensure that, in every state, the policies assign a probability at least ϵ to each action. We can show the proposed method converges using the two time-scale stochastic approximation recipee.

Assumption 2. *We assume the following holds:*

1. *For any policy π , the MDP evolves as an irreducible and aperiodic Markov chain over the state-action space $\mathcal{X} \times \mathcal{A}$. Letting $\mu_{\pi_{\theta_t}}$ denote the stationary distribution obtained at time step t , we assume that $(\mathbf{x}_t, \mathbf{a}_t) \sim \mu_{\pi_{\theta_t}}$.*
2. *$\Sigma = \mathbb{E}_{\mu_{\pi_\theta}} [\phi(\mathbf{x}, \mathbf{a}) \phi^T(\mathbf{x}, \mathbf{a})]$ is non-singular for all π_θ and $\|\phi(x, a)\|_2 \leq 1$, for all (x, a) .*

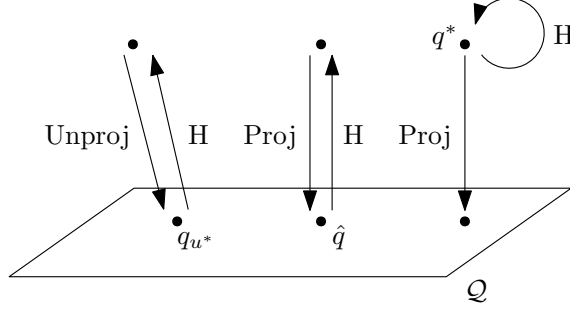


Figure 4.1: Representation of relevant functions and operators. The optimal value function is q^* , and it is the fixed point of the Bellman operator H . The solution of Q -learning is $\hat{q}$, the fixed point of the composition of H and the projection operator Proj onto the function approximation space $\mathcal{Q}$. However, $\hat{q}$ may not exist. Furthermore, $\hat{q}$ does not equal $\text{Proj } q^*$. The regularized solution q_u^* always exists. It does not, however, equal $\text{Proj } q^*$.

Theorem 3. For any initial policy parameters θ_0 and $w_0 \in \mathcal{B}_\rho$,

$$(w_t, \theta_t) \rightarrow (\theta^*, \theta^*),$$

where θ^* is the unique solution to $\theta^* = w^*(\theta^*)$, with $w^*: \mathbb{R}^K \rightarrow \mathbb{R}^K$ such that

$$w^*(\theta) = \mathbb{E}_{\pi_\theta} \left[(r(x, a) + \gamma \phi^T(x', a') w^*(\theta)) \phi(x, a) \right].$$

Proof. The proof of the theorem follows the same lines as the proof of Theorem 2. $\square$

4.3 Discussion

In this section, we proposed regularized alternatives of Q -learning. Now, we expand on the regularization proposed, its implications and connect with related work.

Q -learning with linear function approximation is a stochastic approximation algorithm to solve the fixed-point equation $q = \text{Proj}(Hq)$. Explicitly, the solution to the fixed-point equation verifies, for all state-action pairs, the following:

$$q(x, a) = \phi^T(x, a) \Sigma^{-1} \mathbb{E}_\mu [\phi(x, a) ((Hq)(x, a))]. \quad (25)$$

As debated before, $\text{Proj}H$ may not be contractive and, consequently, a solution $q_{\hat{w}}$ to Equation (25) may not exist and even when it does, convergence to $q_{\hat{w}}$ is not guaranteed. Contrarily, we can guarantee that our proposed regularized versions of Q -learning converge to an existent unique solution. The fixed-point equation CQL tries to solve is the following:

$$q(x, a) = \phi^T(x, a) \mathbb{E}_\mu [\phi(x, a) ((Hq)(x, a))]. \quad (26)$$

Equations (25) and (26) are not the same. The difference between the two is that the normalization matrix Σ^{-1} does not appear in the latter. Therefore, q_{u^*} is the fixed point of the combined operator obtained from an un-normalized “projection”, which we call Unproj , and the Bellman operator. The absence of the matrix Σ^{-1} is key for the emergence of the contraction property of the combined operator. Figure 4.1 adds the regularized fixed point q_{u^*} and the operator Unproj to the figure presented in Section 2.4.2.

It can also be interesting to analyze q_{v^*} . It verifies

$$q_{v^*}(x, a) = \phi^T(x, a) \Sigma^{-1} \mathbb{E}_\mu \left[\phi(x, a) \left((Hq_{u^*})(x, a) \right) \right].$$

The three functions $q_{\hat{w}}$, q_{u^*} and q_{v^*} coincide when $\Sigma = \mathbf{I}$. However, this case does not fall, in general, under our assumptions: even though we can also construct orthogonal features, obtaining a diagonal Σ , since we assume the norm of the features is bounded by 1, the values on the diagonal $\Sigma_{i,i} = \mathbb{E} [\phi_i^2(x, a)]$ are smaller than 1. Still, consider that σ is the minimum eigenvalue of the diagonal Σ . As σ approaches 1, both q_{u^*} and q_{v^*} approach $\hat{q}$. The analysis of the solution obtained by 2TAP follows along the same lines.

More recently, other works provided other regularized alternatives to Q -learning. The solution associated with the method of Zhang et al. [63] solves the fixed-point equation

$$q(x, a) = \phi^T(x, a) (\Sigma + \eta \mathbf{I})^{-1} \mathbb{E}_\mu [\phi(x, a) ((Hq)(x, a))],$$

where $\eta > 0$. The solution associated with the method of Lim et al. [30] solves

$$q(x, a) = \phi^T(x, a) (\xi \Sigma)^{-1} \mathbb{E}_\mu [\phi(x, a) ((Hq)(x, a))],$$

where $\xi > 0$. In both cases, when the parameters η or ξ are sufficiently large, the operators become contractive. Consequently, a unique solution exists and convergence is guaranteed. In our case, the matrix Σ does not appear and contractiveness is guaranteed because the features are assumed small.

Limitations We start by highlighting that all the regularizations just considered, including ours, result in methods that are not scale-invariant, i.e., if the features are changed by a factor, the solution the methods provide, in $\mathcal{Q}$, also change. Additionally, for all the methods referred, including ours, the solutions obtained are not guaranteed to fall close from $\text{Proj } q^*$. Moreover, it is unclear if the solutions are useful for performing control. Chen et al. [13] argues that, with tabular representations, increasing regularization, regardless of which of the methods discussed, is equivalent to lowering the discount factor of the Markov decision process. Sufficiently small discount factors were already known to guarantee convergence [38]. Additionally, except for environments with sparse rewards, lowering the discount factor changes the optimal policy.

5 Learning with non-stationarities

In this section, we extend convergence results of Q -learning to two non-stationary settings. First, we consider linear function approximation with non-stationary features. The setting is motivated by non-linear function approximation with neural networks. Second, we consider tabular representations with non-stationary environments. The setting is motivated by hierarchical reinforcement learning.

5.1 Learning non-linear features as non-stationarity

In the previous section, we provided alternatives to Q -learning with linear function approximation. Our proposed methods are regularized versions of Q -learning. Recently, other works followed the same direction we did and provided regularized methods with convergence guarantees for the case of linear function approximation [30, 63]. In the following, we prove that even if the features are non-stationary, as happens in neural network architectures, regularization can ensure Q -learning methods converge if the features themselves converge. Let us consider neural network architectures as parameterized non-linear functions. A state-action pair (x, a) is the input, and is processed by non-linear features parameterized by $\mathbf{u}$, $\phi_{\mathbf{u}}$. Then, $\phi_{\mathbf{u}}(x, a)$ is passed on to a linear layer parameterized by $\mathbf{v}$. The output is $q_{\mathbf{v}, \mathbf{u}}(x, a)$. We consider $q_{\mathbf{v}, \mathbf{u}} : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$ such that $q_{\mathbf{v}, \mathbf{u}}(x, a) = \phi_{\mathbf{u}}^T(x, a)\bar{\mathbf{v}}$, where $\bar{\cdot} : \mathbb{R}^K \rightarrow \mathcal{B}_\rho$ maps $\mathbf{v}$ to a ball of radius ρ in $\mathbb{R}^K$. We refer to the parameters of the final linear layer, $\mathbf{v} \in \mathbb{R}^K$, as the final parameters, to the parameters of the non-linear hidden layers, $\mathbf{u} \in \mathbb{R}^D$, as the hidden parameters and to $\phi_{\mathbf{u}} : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^K$ as the features. We argue that learning the final parameters $\mathbf{v}$ at a faster time-scale than the hidden parameters $\mathbf{u}$ allows us to decouple the convergence analysis of the two and consequently establish that a regularized version of Q -learning with convergence guarantees for the stationary features case, will also converge if the features are non-stationary but convergent. We define the Q -learning scheme of the final linear layer with the addition of ridge regularization, merging ideas from [30] and [63], as

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \alpha_t \left(r_t + \gamma \max_{a' \in \mathcal{A}} q_{\mathbf{v}_t, \mathbf{u}_t}(x'_t, a') - \xi q_{\mathbf{v}_t, \mathbf{u}_t}(x_t, a_t) \right) \phi_{\mathbf{u}_t}(x_t, a_t) - \alpha_t \eta \mathbf{v}_t,$$

where $\xi, \eta > 0$ are regularization hyper-parameters and the positive learning rates in $\{\alpha_t\}_{t \geq 0}$ are such that $\sum_{t=0}^{\infty} \alpha_t = \infty$ and $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$. Let us assume the features are updated much slower than the final layer through a stochastic approximation scheme with well-behaved noise and that they converge.

Assumption 3. *We assume the following holds:*

1. *The hidden parameters $\mathbf{u}$ are updated according to*

$$\mathbf{u}_{t+1} = \mathbf{u}_t + \beta_t (g(\mathbf{v}_t, \mathbf{u}_t) + \mathbf{n}_{t+1}),$$

2. *The vector field $g : \mathbb{R}^{K+D} \rightarrow \mathbb{R}^D$ is Lipschitz-continuous;*
3. *The sequence of random vectors $\{\mathbf{n}_t\}_{t \in \mathbb{N}}$ has zero mean and finite variance;*
4. *the learning rates in $\{\beta_t\}_{t \in \mathbb{N}_0}$ are such that $\sum_{t=0}^{\infty} \beta_t = \infty$, $\sum_{t=0}^{\infty} \beta_t^2 < \infty$ and $\frac{\beta_t}{\alpha_t} \rightarrow 0$.*
5. *The sequence $\{\mathbf{u}_t\}_{t \in \mathbb{N}_0}$ converges, i.e., that $\mathbf{u}_t \rightarrow \mathbf{u}^*$ for some $\mathbf{u}^* \in \mathbb{R}^D$.*

Theorem 4. *Suppose that Assumption 3 holds and moreover*

1. *For all t , (x_t, a_t, x'_t, r_t) is sampled from $\mathcal{D} = \{(x_i, a_i, x'_i, r_i), i \in \mathbb{N}_0\}$.*
2. *The architecture $\phi : \mathbb{R}^D \times \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^K$ is Lipschitz-continuous on the first argument;*
3. *For all $\mathbf{u} \in \mathbb{R}^D$, the features $\phi_{\mathbf{u}}$ are such that, for all $(x, a) \in \mathcal{X} \times \mathcal{A}$, $\|\phi_{\mathbf{u}}(x, a)\|_2 \leq 1$;*
4. *For all $\mathbf{u} \in \mathbb{R}^D$, the features $\phi_{\mathbf{u}}$ and the distribution μ are such that the $K \times K$ matrix $\Sigma_{\mathbf{u}} := \mathbb{E}[\phi_{\mathbf{u}}(\mathbf{x}, \mathbf{a})\phi_{\mathbf{u}}^T(\mathbf{x}, \mathbf{a})]$ is positive-definite and its minimum eigenvalue is σ .*

Additionally suppose that the regularization parameter $\xi > 1$ is large enough, specifically that $\xi > \frac{\gamma}{\sigma}$; that $\eta > 0$ is small enough, specifically that $\eta < \xi\sigma - \gamma$; and that the radius of the ball $\mathcal{B}_\rho$, $\rho > 0$, is also large enough, specifically that $\rho > \frac{r_{\max}}{\xi\sigma - \gamma - \eta}$.

Then, it holds that $\mathbf{v}_t \rightarrow \mathbf{v}^(\mathbf{u}^*)$, with $\mathbf{v}^* : \mathbb{R}^D \rightarrow \mathbb{R}^K$ such that*

$$\mathbf{v}^*(\mathbf{u}) = (\xi\Sigma_{\mathbf{u}} + \eta\mathbf{I})^{-1} \mathbb{E} \left[\left(r(\mathbf{x}, \mathbf{a}) + \gamma \max_{a' \in \mathcal{A}} q_{\mathbf{v}^*(\mathbf{u}), \mathbf{u}}(\mathbf{x}', a') \right) \phi_{\mathbf{u}}(\mathbf{x}, \mathbf{a}) \right]$$

Proof. The proof follows again from verifying the conditions of Theorem 1. $\square$

5.1.1 Discussion

In this section, we showed how to extend convergence properties of Q -learning from the linear function approximation case to the case when the features are non-stationary but convergent. For our purposes, we considered a regularized Q -learning method. In particular, we showed that if the features are the inner layers of a non-linear neural network, as long as those features converge, the final layer also converges. Convergence of the features, i.e., satisfying Assumption 3, can be obtained through performing stochastic gradient descent on proper loss functions. Specifically, the features can be learned in an unsupervised way, for instance by learning an autoencoder, or in a semi-supervised way, for instance by learning bisimulation metrics. At the same time we are able to preserve the convergence properties of regularized Q -learning methods, we also preserve their drawbacks. Specifically, there is no guarantee that the obtained solution is close to the desired $\text{Proj } q^*$ and that, consequently, it is useful to perform control on Markov decision processes.

5.2 Hierarchical Markov decision processes as non-stationarity

In this section we focus on a non-stationary setting in reinforcement learning where the convergence of Q -learning is unknown. In this case, we consider non-stationary Markov decision processes with respect to the transitions and the rewards. We show that if the Markov decision process converges, Q -learning with tabular representations also converges. The setting is motivated by feudal hierarchical reinforcement learning [15].

Feudal hierarchies In a feudal hierarchy, as introduced by Dayan and Hinton [15], a task is addressed by several agents operating at different levels: higher-level agents act by selecting goals for lower-level agents; the lower-level agents, in turn, act to achieve the goals set by a higher-level agent. While learning a hierarchical policy at both levels simultaneously, an added difficulty of non-stationarity arises [23, 27, 43]. As an example of this non-stationarity, suppose that, in the beginning of training, a high-level agent issues a suitable goal for a low-level agent; if the low-level agent does not achieve its goal,

the high-level agent's decision will not be assigned its due credit. It is not known if the hierarchy sustains a stable behavior. We start by formalizing the hierarchy as a pair of Markov decision processes. Then, we show the processes are stable. Finally, we prove the convergent behavior of Q -learning, resorting to the stability result. Specifically, we prove that Q -learning at the high-level converges if the low-level policy converges. Consider a shallow Markov decision process $\mathcal{M} := (\mathcal{X}, \mathcal{A}, P, r)$ as described in Section 2 and a discount factor γ . We use superscripts h and l to denote the *high-level* and *low-level* components of a two-level hierarchy, respectively. We define a two-level hierarchy as a pair $(\mathcal{M}^h, \mathcal{M}^l)$, with $\mathcal{M}^h(\pi^l) := (\mathcal{X}, \Omega, P^h(\pi^l), r^h(\pi^l), \gamma^h)$ and $\mathcal{M}^l := (\mathcal{X} \times \Omega, \mathcal{A}, P^l, r^l, 1)$. Above, on top of the notation already established for Markov decision processes, we used Ω to denote the action space of the high-level MDP and, simultaneously, one of the components of the state space of the low-level MDP. We also call Ω the *goal* space. On the high-level process $\mathcal{M}^h$, we note the dependency of the high-level transition probabilities P^h and reward r^h on the policy π^l of the agents at the lower-level. On the low-level process, the reward function $r^l : (\mathcal{X} \times \Omega) \times \mathcal{A} \rightarrow \mathbb{R}$ can be any.

Assumption 4. *The stopping time of a low-level episode, τ , is a random variable supported in $\mathbb{N}_0$ with probability mass function $\xi(t) := P(\tau = t) = (1 - \gamma)\gamma^t$.*

Proposition 1. *Consider the Markov decision processes $\mathcal{M}$, $\mathcal{M}^h$ and $\mathcal{M}^l$ and a converging sequence of low-level policies $\pi_t^l \rightarrow \pi^l$, $t \in \mathbb{N}$. For each high-level policy π^h , it holds that:*

1. $P^h(\pi_t^l) \rightarrow P^h(\pi^l)$;
2. $r^h(\pi_t^l) \rightarrow r^h(\pi^l)$;
3. $v_{\pi^h}^h(\pi_t^l) \rightarrow v_{\pi^h}^h(\pi^l)$;
4. $q_{\pi^h}^h(\pi_t^l) \rightarrow q_{\pi^h}^h(\pi^l)$.

Proof. We leave the idea for the proof. We start by noting that, in metric spaces, if an element $f_t \rightarrow f^*$, we have that $G(f_t) \rightarrow G(f^*)$ if and only if G is continuous. Therefore, the proposition equivalently states that, for every high-level policy π^h , P^h , r^h , $V_{\pi^h}^h$ and $Q_{\pi^h}^h$ are continuous on the low-level policies π^l . We prove the continuity of each component. $\square$

Convergence We prove that Q -learning converges in non-stationary but convergent Markov decision processes by showing that it converges on the high level of a feudal hierarchy. We consider the low-level learns using any learning algorithm at any learning rate, as long as it converges, and the high-level uses Q -learning. We show that, even though the algorithms operate over a non-stationary Markov decision process, convergence still holds. We introduce some notation. $r_t(x, \omega)$ is a reward sample distributed according to $r^h(\pi_t^l)(x, \omega)$; $y_t(x, \omega)$ a next state sample distributed according to $P^h(\pi_t^l)(x, \omega)$; $q_t^{*,h}$ is the optimal value function of the high-level Markov decision process $\mathcal{M}^h(\pi_t^l)$, where π_t^l is the low-level policy at time step t . Consider a sequence of low-level policies π_t^l converging to the optimal low-level policy $\pi^{*,l}$. Q -learning on the high level performs the updates

$$q_{t+1}^h(x, \omega) = q_t^h(x, \omega) + \alpha_t(x, \omega) \left(r_t(x, \omega) + \gamma^{\frac{1}{\gamma}} \max_{\omega'} q_t^h(y_t(x, \omega), \omega') - q_t^h(x, \omega) \right),$$

Theorem 5. *Let the high-level learning rates $\alpha_t : \mathcal{X} \times \Omega \rightarrow [0, 1]$ be such that, for all (x, ω) , $\sum_t \alpha_t(x, \omega) = \infty$ and $\sum_t \alpha_t^2(x, \omega) < \infty$. Then, Q -learning converges to $q^{*,h}$.*

Proof. The proof follows from verifying the conditions on an established convergence result on stochastic processes [53], which we reproduce. Consider the random iteration

$$\Delta_{t+1}(z) = (1 - \alpha_t(z))\Delta_t(z) + \alpha_t(z)F_t(z),$$

where $z \in \mathcal{Z}$ and $\mathcal{Z}$ is finite. Consider also the sequence of sigma-fields $\mathcal{F}_t$ such that $(\alpha_t(z), \Delta_t(z), F_t(z)) \in \mathcal{F}_t$. The sequence $\{\Delta_t\}_{t \in \mathbb{N}_0}$ converges to 0 under the following conditions, for each z in $\mathcal{Z}$:

1. $\|\mathbb{E}[F_t(z) \mid \mathcal{F}_t]\| \leq \gamma\|\Delta_t\| + c_t, c_t \rightarrow 0$;
2. $\text{var}(F_t(z) \mid \mathcal{F}_t) \leq K(1 + \|\Delta_t\|)^2$.

To apply the convergence result, we consider the stochastic processes

$$\begin{aligned}\Delta_t(x, \omega) &= q_t^h(x, \omega) - q^{*,h}(x, \omega) \\ F_t(x, \omega) &= r_t(x, \omega) + \gamma^h \max_{a'} q_t^h(y_t(x, \omega), \omega') - q^{*,h}(x, \omega)\end{aligned}$$

and show that they satisfy the conditions referred. □

5.2.1 Discussion

Our work is not the first that looks at the asymptotic behaviour of reinforcement learning on non-stationary Markov decision processes [21, 22, 35, 28]. In our case, we establish a convergence result for Q -learning on the high-level of a feudal hierarchy if the low-level is convergent. Equivalently, we prove convergence of Q -learning on convergent sequences of Markov decision processes. The control result is not, in general, a particular case of the one from [35], as optimal q -values are not independent of the history of the process throughout the interaction. In fact, the optimal Q -values are time dependent, even though they converge to a time-independent $q^{*,h}$ if the low-level policies converge. Additionally, we do not impose ergodicity of the Markov decision processes.

6 Future work

Here, we expand on the two research directions we intend to follow next.

6.1 Between gradients and semi-gradients

The convergence results we established for Q -learning with function approximation rely on regularization of the parameters of the function approximator or of the Q -values. The regularization imposes a bias on the solutions obtained by the resulting Q -learning schemes, in the sense that the solutions obtained are not the ones that the original Q -learning aimed at, i.e., the fixed point of $\text{Proj } H$. One way to look at Q -learning is that it performs stochastic semi-gradient descent on the Bellman error $l : \mathbb{R}^K \rightarrow \mathbb{R}$ defined as

$$l(\mathbf{w}) = \frac{1}{2} \mathbb{E} \left[((Hq_{\mathbf{w}})(\mathbf{x}, \mathbf{a}) - q_{\mathbf{w}}(\mathbf{x}, \mathbf{a}))^2 \right].$$

By “semi-gradient”, we mean that the target $Hq_{\mathbf{w}}$ is not differentiated. The semi-gradient is

$$\nabla l(\mathbf{w}) = -\mathbb{E} [((Hq_{\mathbf{w}})(\mathbf{x}, \mathbf{a}) - q_{\mathbf{w}}(\mathbf{x}, \mathbf{a})) \nabla q_{\mathbf{w}}(\mathbf{x}, \mathbf{a})],$$

resulting in expected updates of the form

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha_t \nabla l(\mathbf{w}).$$

The full gradient is

$$\nabla l(\mathbf{w}) = \nabla l(\mathbf{w}) + \mathbb{E} [((Hq_{\mathbf{w}})(\mathbf{x}, \mathbf{a}) - q_{\mathbf{w}}(\mathbf{x}, \mathbf{a})) ((\nabla (Hq_{\mathbf{w}}))(\mathbf{x}, \mathbf{a}))],$$

with full gradient descent updates

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha_t \nabla l(\mathbf{w}).$$

The full gradient update would have stronger convergence properties [39]. Other works explored the direction of full-gradient Q -learning, as discussed in Section 3.5. Those methods are convergent but the limit solution may not be good, since they can minimize the Bellman objective strictly locally which, in general, has no relation with the quality of the resulting control policy. Additionally, full gradient requires double sampling from each state and action to obtain two unbiased samples of the next state. Along this future work direction, we intend to propose a method that avoids both issues just mentioned.

Local minima issue The method that results from performing true stochastic gradient descent on the Bellman error would be convergent with probability 1 [39, 3]. Sutton and Barto [54] discuss reasons for not using the full-gradient descent scheme. The discussion mostly blames the fact that such scheme could converge to undesirable Q -values. Since the only global minimum of the Bellman error is identically zero if we assume sufficient representation capacity of a neural network, undesirable Q -values can only appear if convergence happens to strictly local minima of the Bellman error. Kawaguchi and Kaelbling [24] proposes a way to guarantee, theoretically, that the limit solution encountered by neural networks globally minimizes the objective. Therefore we intend to use that recent technique to ensure that convergence of the full-gradient Q -learning scheme with neural networks happens to global minima [24]. While it eliminates the possibility of converging to strictly local minima, it is not guaranteed that such limit solution exists, however.

The double sampling issue Another issue with the full-gradient method is that full-gradient descent on the Bellman error implies the estimation of a product of two expected values. Specifically, in the full gradient update, the term

$$\mathbb{E} [((Hq_w)(x, a)) \cdot ((\nabla (Hq_w))(x, a))]$$

appears, requiring two samples of the next state to obtain an unbiased sample of the product inside the expectation.⁸ To do so, and if the environment is stochastic with respect to the transitions, it is necessary to get two independent action samples from each state the agent visits. While this issue may not be harmful in easily simulated environments, it may be so if we want our agent to learn online and can not easily simulate two transitions from the same state. However, we intend to show that if we keep a separate set of parameters to estimate the variance on the transitions and update it at a slower time-scale we can ensure convergence of the full-gradient descent method without the issue of double-sampling. Other works have also provided methods to deal with the double-sampling issue [2, 14, 33, 42]. We intend to replicate the technique applied by Maei et al. [32] for the double-sampling issue that also exists in gradient-TD methods introduced in Section 3.1.2.

6.2 Multi-Bellman operator

In Section 4.1, we proposed convergent Q -learning methods with linear function approximation. The methods include a regularization that guarantees the existence and uniqueness of solution and convergence. However, the solution may fall far from the desired $\text{Proj } q^*$. Along the research direction we propose here, we aim at developing a Q -learning method that converges to a solution that is closer to q^* .

Fixed-point analysis A solution to Q -learning with linear function approximation must be the fixed point of the projected Bellman operator, verifying the equation

$$q = \text{Proj} (Hq). \quad (27)$$

In general, the composed operator $\text{Proj } H$ is not contractive and a solution to the fixed-point equation (27) does not exist. Even when the solution exists, it is not guaranteed that successively applying $\text{Proj } H$ generates a sequence that converges it. To circumvent this shortcoming, we introduce the multi-Bellman operator H^n . The multi-Bellman operator results from applying the Bellman operator multiple times. First, we have that

$$(H^1 q)(x_0, a_0) = \mathbb{E} \left[r(x_0, a_0) + \gamma \max_{a_1 \in \mathcal{A}} q(x_1, a_1) \right].$$

Then, we have that

$$(H^2 q)(x_0, a_0) = \mathbb{E} \left[r(x_0, a_0) + \gamma \max_{a_1 \in \mathcal{A}} \mathbb{E} \left[r(x_1, a_1) + \gamma \max_{a_2 \in \mathcal{A}} q(x_2, a_2) \right] \right].$$

More generally, we have that

$$(H^n q)(x_0, a_0) = \mathbb{E} \left[r(x_0, a_0) + \gamma \max_{a_1 \in \mathcal{A}} \mathbb{E} \left[r(x_1, a_1) + \gamma \max_{a_2 \in \mathcal{A}} \mathbb{E} \left[\cdots + \gamma \max_{a_n \in \mathcal{A}} q(x_n, a_n) \right] \right] \right].$$

We start by noting that, since H^1 is a contraction of factor γ , the operator H^n is a contraction of factor γ^n . While unsurprising, this remark can have profound implications on the use of linear function approximation. Specifically, we are able to prove the following result.

⁸The Bellman operator of a function at a state-action pair is an expectation over the next state.

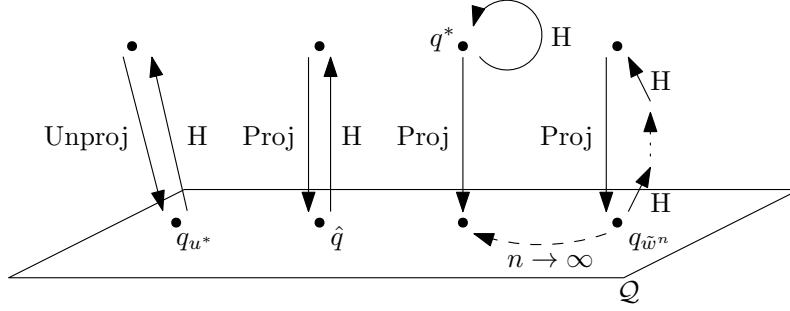


Figure 6.1: Representation of relevant functions and operators. The optimal value function is q^* , and it is the fixed point of the Bellman operator H . The solution of Q -learning is $\hat{q}$, the fixed point of the composition of H and the projection operator Proj onto the function approximation space $\mathcal{Q}$. However, $\hat{q}$ may not exist. Furthermore, $\hat{q}$ does not equal $\text{Proj } q^*$. The regularized solution q_u^* always exists. It does not, however, equal $\text{Proj } q^*$. For sufficiently large n , the fixed point of $\text{Proj } H^n$, $q_{\tilde{w}^n}$, exists. As n grows, $q_{\tilde{w}^n} \rightarrow \text{Proj } q^*$.

Theorem 6. *There exists $N \in \mathbb{N}$ such that, for all $n \geq N$, $\text{Proj } H^n$ is a contraction.*

Proof. The proof follows from observing that Proj is Lipschitz-continuous, with constant $c_{\phi, \mu}$ and that H^n is a contraction of factor γ^n . It follows that, for sufficiently large n , the composition of the two operators is contractive. $\square$

As a consequence, for sufficiently large n , there exists a unique solution to

$$q = \text{Proj } (H^n q). \quad (28)$$

and successively applying the operator $\text{Proj } H^n$ generates a sequence that converges to the solution. Furthermore, as n increases, the fixed point becomes closer to $\text{Proj } q^*$.

Proposition 2. *Let $q_{\tilde{w}^n}$ denote the solution to Equation (28), for $n \geq N$. As $n \rightarrow \infty$,*

$$q_{\tilde{w}^n} \rightarrow \text{Proj } q^*.$$

Proof. Since, $q^* = Hq^*$, it also holds that $q^* = H^n q^*$ for all n and we can start by writing

$$\|\text{Proj } q^* - q_{\tilde{w}^n}\|_{\infty} = \|\text{Proj } (H^n q^*) - \text{Proj } (H^n q_{\tilde{w}^n})\|_{\infty}.$$

Recalling that Proj is Lipschitz-continuous with constant $c_{\phi, \mu}$, it follows that

$$\begin{aligned} \|\text{Proj } q^* - q_{\tilde{w}^n}\|_{\infty} &\leq c_{\phi, \mu} \|H^n q^* - H^n q_{\tilde{w}^n}\|_{\infty} \\ &\leq c_{\phi, \mu} \gamma^n \|q^* - q_{\tilde{w}^n}\|_{\infty}. \end{aligned}$$

Since the right-hand side vanishes to zero as n grows, the conclusion follows. $\square$

Figure 6.1 adds $q_{\tilde{w}^n}$ and the insight from Proposition 2 to the figure presented in Section 4.3. We have established a fixed point equation does have a solution. Now, we intend to develop a stochastic approximation algorithm that converges to that solution. We describe our first attempt below. In the future, we intend to improve its limitations.

Stochastic approximation Let us now propose a stochastic approximation algorithm for $\text{Proj } H^n$. We propose to perform the multi Q -learning update

$$\mathbf{w}_{t+1}^n = \mathbf{w}_t^n + \alpha_t [\phi(x_t, a_t) (\tau_{t+1}^n - q_{\mathbf{w}_t^n}(x_t, a_t))],$$

where the target τ_{t+1}^n is a sample for $(H^n q_{\mathbf{w}_t^n})(x_t, a_t)$ such that

$$\tau_{t+1}^n = r_t + \gamma \max_{\bar{a} \in \mathcal{A}^n} [r_{t+1} + \gamma^1 r_{t+2} + \dots + \gamma^{n-1} q_{\mathbf{w}_t^n}(x_{t+n}, a_{t+n})],$$

with $\bar{a} = (a_{t+1}, a_{t+2}, \dots, a_{t+n})$ and r_{t+m} and x_{t+m+1} are obtained by performing action a_{t+m} on state x_{t+m} for m between 1 and n . In practice, at time step t , instead of building a 1-step greedy target, multi Q -learning builds an n -step target that is obtained by trying every action on every state encountered along an n -step trajectory starting at x_{t+1} .

6.2.1 Discussion

The method proposed has several limitations that we intend to address in the continuation of this research. We expand on each of the limitations and solution directions below.

Convergence Unfortunately, since the maximization of the expected value is not the expected value of the maximization, the updates of the proposed multi Q -learning are biased and we can only guarantee convergence to true the fixed point of equation (28) if the environment is deterministic. The reason for the bias is that we only sample one time each action on each state. Convergence to the true solution holds in the limit as the number of action samples at each state grows and the value estimates are averaged. We intend to quantize the error introduced in the solution of multi Q -learning in response to considering only a finite number of action samples per action in each state.

Model Multi Q -learning requires a model from which to sample every action on every step along n -step trajectories. It is not the case that such known model is always available in applications. There are several techniques for learning a model of the environment. For example, we can learn the model previously or concurrently to reinforcement learning; the model can be exact or inexact; the model can be parametric or non-parametric. Multi Q -learning can be combined with any of the mentioned model-based approaches in order to remove the need for a known model. We intend to show that, with the use of two time-scales, we can concurrently learn a model of the environment and the estimate for the Q -values while maintaining convergence guarantees.

Complexity Multi Q -learning plans by performing every action on every reached state along trajectories of n steps. Thus, the computational cost of performing an update grows exponentially with n , where the base for the exponent is the size of the action space. In case only some actions are performed on each state, we cannot show convergence to the solution to an appropriate fixed-point equation, even when the environment is deterministic. Nevertheless, we expect that, in some cases, the computational benefits of not performing every action on every state to outweigh the theoretical comfort. We intend to quantize the error introduced in the solution of multi Q -learning in response to considering only a subset of the action space in each state.

7 Conclusion

This proposed thesis is concerned with exploring Q -learning algorithms to be used with function approximation that are provably convergent and such that their limit solution is good. Our methods and proofs mostly rely on two time-scale stochastic approximation. We start by providing an alternative to Q -learning with linear function approximation that is regularized. We then move to settings with non-stationarities, proving that a regularized version of Q -learning converges if the features are non-stationary but convergent and that Q -learning with tabular representations converge in non-stationary environments, by focusing on a feudal hierarchical setting. We intend to finish our research by presenting two Q -learning methods to be used with non-linear function approximation, specifically neural networks. First, we aim at providing a Q -learning method that performs full-gradient descent updates and such that any limit solution is globally optimal. Second, we intend to explore a multi-Bellman operator which has stronger convergence properties than the Bellman operator, and an associated multi Q -learning algorithm.

References

- [1] Naman Agarwal, Syomantak Chaudhuri, Prateek Jain, Dheeraj Nagaraj, and Praneeth Netrapalli. Online target q-learning with reverse experience replay: Efficiently finding the optimal policy for linear mdps. *arXiv preprint arXiv:2110.08440*, 2021.
- [2] András Antos, Csaba Szepesvári, and Rémi Munos. Learning near-optimal policies with bellman-residual minimization based fitted policy iteration and a single sample path. *Machine Learning*, 71(1):89–129, 2008.
- [3] Konstantin E. Avrachenkov, Vivek S. Borkar, Hars P. Dolhare, and Kishor Patil. Full gradient dqn reinforcement learning: a provably convergent scheme. In *Modern Trends in Controlled Stochastic Processes*, pages 192–220. Springer, 2021.
- [4] Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In *International Conference on Machine Learning*, pages 30–37, 1995.
- [5] Andrew G Barto, Richard S Sutton, and Charles W Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics*, (5):834–846, 1983.
- [6] Joan Bas-Serrano, Sebastian Curi, Andreas Krause, and Gergely Neu. Logistic q-learning. In *International Conference on Artificial Intelligence and Statistics*, pages 3610–3618. PMLR, 2021.
- [7] Richard Bellman. Dynamic programming. *Science*, 153(3731):34–37, 1966.
- [8] Shalabh Bhatnagar, Richard Sutton, M. Ghavamzadeh, and M. Lee. Incremental natural actor-critic algorithms. In *Advances in Neural Information Processing Systems*, pages 105–112, 2007.
- [9] Vivek Borkar. *Stochastic Approximation: A Dynamical Systems Viewpoint*. Cambridge University Press, 2008.
- [10] Justin A. Boyan and Andrew W. Moore. Generalization in reinforcement learning: Safely approximating the value function. In *Advances in Neural Information Processing Systems*, pages 369–376, 1995.
- [11] Qi Cai, Zhuoran Yang, Jason D Lee, and Zhaoran Wang. Neural temporal-difference learning converges to global optima. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL <https://proceedings.neurips.cc/paper/2019/file/98baeb82b676b662e12a7af8ad9212f6-Paper.pdf>.
- [12] Diogo S. Carvalho, Francisco S. Melo, and Pedro A. Santos. A new convergent variant of q-learning with linear function approximation. *Advances in Neural Information Processing Systems*, 33, 2020.
- [13] Zaiwei Chen, John Paul Clarke, and Siva Theja Maguluri. Target network and truncation overcome the deadly triad in q-learning. *arXiv preprint arXiv:2203.02628*, 2022.

- [14] Bo Dai, Albert Shaw, Lihong Li, Lin Xiao, Niao He, Zhen Liu, Jianshu Chen, and Le Song. Sbeed: Convergent reinforcement learning with nonlinear function approximation. In *International Conference on Machine Learning*, pages 1125–1134. PMLR, 2018.
- [15] Peter Dayan and Geoffrey E. Hinton. Feudal reinforcement learning. In S. Hanson, J. Cowan, and C. Giles, editors, *Advances in Neural Information Processing Systems*, volume 5. Morgan-Kaufmann, 1993.
- [16] Simon S Du, Jianshu Chen, Lihong Li, Lin Xiao, and Dengyong Zhou. Stochastic variance reduction methods for policy evaluation. In *International Conference on Machine Learning*, pages 1049–1058. PMLR, 2017.
- [17] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In *International Conference on Machine Learning*, pages 1407–1416. PMLR, 2018.
- [18] Amir-massoud Farahmand. Regularization in reinforcement learning. 2011.
- [19] Geoffrey J. Gordon. Reinforcement learning with function approximation converges to a region. In *Advances in Neural Information Processing Systems*, pages 1040–1046, 2001.
- [20] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870. PMLR, 2018.
- [21] Marcus Hutter. Extreme state aggregation beyond markov decision processes. *Theoretical Computer Science*, 650:73–91, 2016.
- [22] Marcus Hutter, Samuel Yang-Zhao, and Sultan J. Majeed. Conditions on features for temporal difference-like methods to converge. *arXiv preprint arXiv:1905.11702*, 2019.
- [23] Yiding Jiang, Shixiang (Shane) Gu, Kevin P Murphy, and Chelsea Finn. Language as an abstraction for hierarchical deep reinforcement learning. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [24] Kenji Kawaguchi and Leslie Kaelbling. Elimination of all bad local minima in deep learning. In *International Conference on Artificial Intelligence and Statistics*, pages 853–863. PMLR, 2020.
- [25] Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems*, pages 1008–1014. Citeseer, 2000.
- [26] Chandrashekar Lakshminarayanan and Shalabh Bhatnagar. A stability criterion for two timescale stochastic approximation schemes. *Automatica*, 79:108–114, 2017.
- [27] Andrew Levy, George Konidaris, Robert Platt, and Kate Saenko. Learning multi-level hierarchies with hindsight. *Proceedings of the 7th International Conference on Learning Representations*, 2017.

- [28] Lihong Li, Thomas J Walsh, and Michael L Littman. Towards a unified theory of state abstraction for mdps. *ISAIM*, 4:5, 2006.
- [29] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- [30] Han-Dong Lim, Do Wan Kim, and Donghwan Lee. Regularized q-learning. *arXiv preprint arXiv:2202.05404*, 2022.
- [31] Aleksandr Mikhailovich Lyapunov. The general problem of the stability of motion. *International journal of control*, 55(3):531–534, 1992.
- [32] Hamid R. Maei, Csaba Szepesvari, Shalabh Bhatnagar, Doina Precup, David Silver, and Richard Sutton. Convergent temporal-difference learning with arbitrary smooth function approximation. In *Advances in Neural Information Processing Systems*, pages 1204–1212, 2010.
- [33] Hamid R. Maei, Csaba Szepesvári, Shalabh Bhatnagar, and Richard S. Sutton. Toward off-policy learning control with function approximation. In *International Conference on Machine Learning*, pages 719–726, 2010.
- [34] Sridhar Mahadevan, Bo Liu, Philip Thomas, Will Dabney, Steve Giguere, Nicholas Jacek, Ian Gemp, and Ji Liu. Proximal reinforcement learning: A new theory of sequential decision making in primal-dual spaces. *arXiv preprint arXiv:1405.6757*, 2014.
- [35] Sultan J. Majeed and Marcus Hutter. On q-learning convergence for non-markov decision processes. In *IJCAI*, pages 2546–2552, 2018.
- [36] Alan S Manne. Linear programming and sequential decisions. *Management Science*, 6(3):259–267, 1960.
- [37] Francisco S Melo and M Isabel Ribeiro. Q-learning with linear function approximation. In *Learning Theory: 20th Annual Conference on Learning Theory, COLT 2007, San Diego, CA, USA; June 13-15, 2007. Proceedings 20*, pages 308–322. Springer, 2007.
- [38] Francisco S. Melo, Sean Meyn, and M. Isabel Ribeiro. An analysis of reinforcement learning with function approximation. In *International Conference on Machine Learning*, pages 664–671, 2008.
- [39] Panayotis Mertikopoulos, Nadav Hallak, Ali Kavis, and Volkan Cevher. On the almost sure convergence of stochastic gradient descent in non-convex problems. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1117–1128. Curran Associates, Inc., 2020.
- [40] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

- [41] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, pages 1928–1937. PMLR, 2016.
- [42] Rémi Munos and Csaba Szepesvári. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9(5), 2008.
- [43] Ofir Nachum, Shixiang (Shane) Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.
- [44] Dirk Ormoneit and Šaunak Sen. Kernel-based reinforcement learning. *Machine learning*, 49(2):161–178, 2002.
- [45] Theodore J. Perkins and Doina Precup. A convergent form of approximate policy iteration. In *Advances in neural information processing systems*, pages 1627–1634, 2003.
- [46] Alexandre Piché, Joseph Marino, Gian Maria Marconi, Christopher Pal, and Mohammad Emtiyaz Khan. Beyond target networks: Improving deep q -learning with functional regularization. *arXiv preprint arXiv:2106.02613*, 2021.
- [47] Martil L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2005.
- [48] Herbert Robbins and Sutton Monro. A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [49] Ehsan Saleh and Nan Jiang. Deterministic bellman residual minimization. In *Proceedings of Optimization Foundations for Reinforcement Learning Workshop at NeurIPS*, 2019.
- [50] Bruno Scherrer. Should one compute the temporal difference fix point or minimize the bellman residual? the unified oblique projection view. *arXiv preprint arXiv:1011.4362*, 2010.
- [51] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [52] Satinder Singh, Tommi Jaakkola, and Michael Jordan. Reinforcement learning with soft state aggregation. *Advances in Neural Information Processing Systems*, 7, 1994.
- [53] Satinder Singh, Tommi Jaakkola, Michael L Littman, and Csaba Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine learning*, 38(3):287–308, 2000.
- [54] Richard Sutton and Andrew Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [55] Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, pages 1057–1063, 2000.

- [56] Csaba Szepesvári and William D Smart. Interpolation-based q-learning. In *International Conference on Machine Learning*, page 100, 2004.
- [57] John N. Tsitsiklis and Benjamin Van Roy. Feature-based methods for large scale dynamic programming. *Machine Learning*, 22:59–94, 1996.
- [58] Zhikang T. Wang and Masahito Ueda. A convergent and efficient deep q network algorithm. *arXiv preprint arXiv:2106.15419*, 2021.
- [59] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3): 279–292, 1992.
- [60] Pan Xu and Quanquan Gu. A finite-time analysis of q-learning with neural network function approximation. In *International Conference on Machine Learning*, pages 10555–10565. PMLR, 2020.
- [61] Shuyu Yin, Tao Luo, Peilin Liu, and Zhi-Qin John Xu. An experimental comparison between temporal difference and residual gradient with neural network approximation. *arXiv preprint arXiv:2205.12770*, 2022.
- [62] Shangdong Zhang, Wendelin Boehmer, and Shimon Whiteson. Deep residual reinforcement learning. *arXiv preprint arXiv:1905.01072*, 2019.
- [63] Shangdong Zhang, Hengshuai Yao, and Shimon Whiteson. Breaking the deadly triad with a target network. In *International Conference on Machine Learning*, pages 12621–12631. PMLR, 2021.